VERIQTA

VOLUME 3

ADVANCED DEVOPS INTERVIEW QUESTIONS

VOLUME 3: AWS, CLOUD ARCHITECTURE & ENTERPRISE INFRASTRUCTURE

Real-world. Production-focused. Senior Engineer Level.

AWS EXPERTISE
Deep dive into AWS services, architectures and best practices

CLOUD ARCHITECTURE
Design scalable, secure and highly available enterprise solutions

ENTERPRISE INFRASTRUCTURE
Multi-account, networking, security, governance and operational excellence

DEVOPS & AUTOMATION
CI/CD, Infrastructure as Code, GitOps, monitoring and production automation

PRODUCTION FOCUS
Real incidents, troubleshooting, system design, and performance optimization

INTERVIEW READY
200+ advanced interview questions with detailed engineer-level answers



WHAT YOU'LL MASTER:

- ✓ AWS Services Deep Dive
- ✓ Cloud Architecture & Design
- ✓ Enterprise Infrastructure
- ✓ Security & Compliance
- ✓ High Availability & DR
- ✓ Performance Optimization
- ✓ Real Production Scenarios
- ✓ Troubleshooting Mastery
- ✓ System Design & Best Practices

THINK LIKE A SOLUTION ARCHITECT.
BUILD LIKE A LEADER.

SENIOR LEVEL DEPTH | REAL-WORLD SCENARIOS | PRODUCTION FOCUSED | INTERVIEW READY | CAREER ACCELERATOR

VERIQTA

@veriqta_ | Veriqta | t.me/veriqta | veriqta.com

MASTER CLOUD. ENGINEER SYSTEMS. LEAD AT SCALE.

PAGE 01 OF 20

VERIQTA

@veriqta_

AWS GLOBAL INFRASTRUCTURE & PRODUCTION ARCHITECTURE

SENIOR INTERVIEW TOPICS

Build for scale. Design for failure. Operate with excellence.

ENGINEER MINDSET

- ☑ Design for failure
- ☑ Automate everything
- ☑ Measure & improve
- ☑ Secure by default
- ☑ Think long term
- ☑ Own the impact

TOPICS COVERED ON THIS PAGE

1. AWS Global Infrastructure
2. Regions and Availability Zones
3. Edge Locations
4. High Availability Design
5. Fault Isolation
6. Shared Responsibility Model
7. Enterprise Cloud Architecture



AWS BY THE NUMBERS

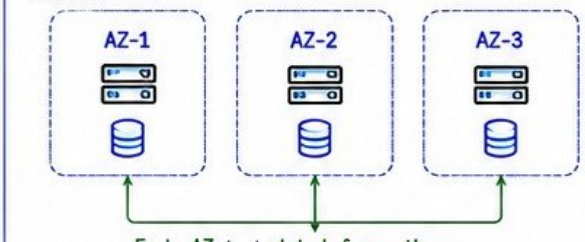
- **31** Regions
- **99+** Availability Zones
- **400+** Edge Locations
- **245+** Countries & Territories
- **Millions** of active customers

1 AWS GLOBAL INFRASTRUCTURE

- AWS runs one of the most secure, scalable and reliable cloud infrastructures in the world.
- Built on a global network of data centers, connected by high-speed, low-latency fiber networks.
- Designed for durability, performance and security at massive scale.

2 REGIONS AND AVAILABILITY ZONES

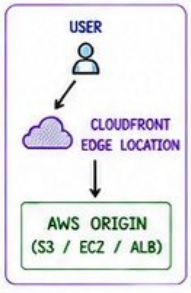
- A Region is a physical location in the world.
- Each Region has multiple, isolated Availability Zones (AZs).
- AZs are separated by distance, power, cooling and networking.
- Use multiple AZs to achieve high availability and fault tolerance.



3 EDGE LOCATIONS

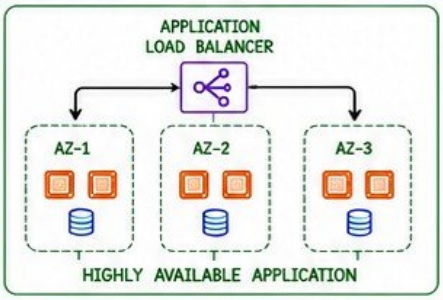
Edge locations are part of Amazon CloudFront.

- Content is cached closer to users to reduce latency and improve performance.
- Protects against DDoS attacks and improves availability.
- Used for CDN, static content, streaming and API acceleration.



4 HIGH AVAILABILITY DESIGN

- Distribute resources across multiple AZs.
- Eliminate single points of failure.
- Use load balancers, Auto Scaling and managed services.
- Design patterns:
  - ☑ Multi-AZ Architecture
  - ☑ Active-Passive
  - ☑ Active-Active
  - ☑ Multi-Region



5 FAULT ISOLATION

Failures happen. Isolation limits blast radius.

- AZs are isolated at hardware, network and power levels.
- Use separate subnets, route tables, security groups and accounts.
- Apply chaos engineering and failure testing.

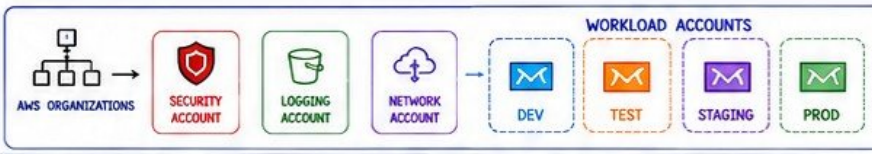


6 SHARED RESPONSIBILITY MODEL

AWS (RESPONSIBLE FOR)	CUSTOMER (RESPONSIBLE FOR)
Global Infrastructure	Data & Content
Hardware, Software, Networking	IAM & Access Management
Facilities, Regions, AZs	OS, Middleware, Applications
Security of the Cloud	Security in the Cloud

7 ENTERPRISE CLOUD ARCHITECTURE

- Build secure, scalable and governed cloud environments.
- Use AWS Organizations for multi-account management.
- Separate workloads by environment: Dev, Test, Staging, Production.
- Implement landing zones, guardrails and centralized services.



PRO TIP
The best architectures are boring, resilient and observable.

GOLDEN RULE
Design for failure.
Expect the unexpected.

REMEMBER
Scale is easy.
Resilience is hard.

PAGE 02 OF 20

VERIQTA

@veriqta_

EC2 PRODUCTION INTERVIEW QUESTIONS

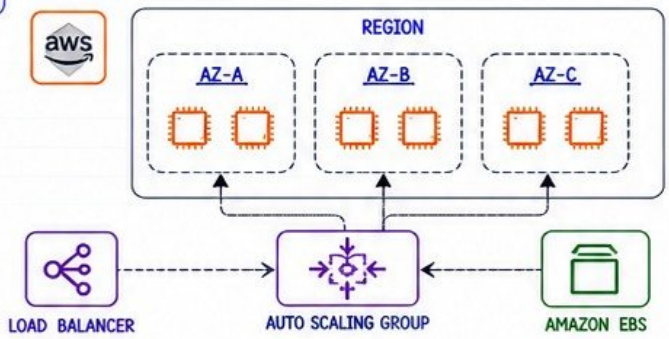
SENIOR INTERVIEW TOPICS

Design resilient. Scale intelligently. Optimize continuously.

REMEMBER
- ☑ Right instance
- ☑ Right size
- ☑ Right capacity
- ☑ Right availability
- ☑ Right cost

TOPICS COVERED ON THIS PAGE

1. EC2 Instance Families
2. Auto Scaling Groups
3. Launch Templates
4. Placement Groups
5. Instance Lifecycle
6. Capacity Planning
7. Production Optimization



KEY EC2 BENEFITS

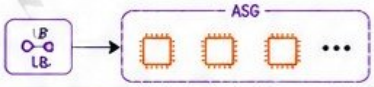
- Secure & isolated
- Highly available
- Scalable
- Flexible
- Cost effective
- Wide instance choice

1 EC2 INSTANCE FAMILIES

FAMILY	BEST FOR	EXAMPLES
General Purpose	Balanced compute, memory & networking	T4g, M6i, M7i
Compute Optimized	High performance computing	C6i, C7i
Memory Optimized	In-memory databases, analytics	R6i, X2idn
Storage Optimized	High IOPS, large datasets	I4i, D3
Accelerated Computing	GPU / ML / AI workloads	P4d, G5, Trn1
Graviton (ARM)	Cost-effective, high performance	T4g, M7g, C7g

2 AUTO SCALING GROUPS

- Automatically scales EC2 instances based on demand.
- Maintains application availability & fault tolerance.
- Components:
  - ☑ Launch Template / Configuration
  - ☑ Desired Capacity
  - ☑ Min / Max Capacity
  - ☑ Scaling Policies
  - ☑ Health Checks
- Use CloudWatch metrics for trigger (CPU, Requests, Queue depth etc).



3 LAUNCH TEMPLATES

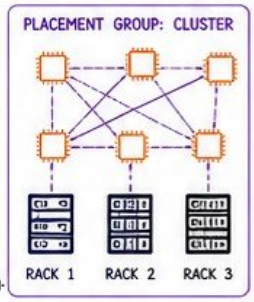
- Defines instance configuration (AMI, Instance type, SG, IAM Role, User data, EBS, etc).
- Versioned & reusable.
- Best practice for ASG and consistent deployments.

LAUNCH TEMPLATE EXAMPLE

```
Image ID    : ami-0abc123456789
Instance    : t3.large
Key Pair    : veriqta-key
SG          : sg-web
IAM Role    : ec2-role
User Data   : install.sh
EBS         : 50 GiB gp3
```

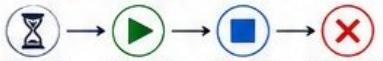
4 PLACEMENT GROUPS

- Control how instances are placed in underlying hardware.
- Types:
  - ☑ Cluster – Low latency, high network performance.
  - ☑ Spread – Maximize hardware spread (rack-level).
  - ☑ Partition – Large distributed workloads (Hadoop, Cassandra).
- Improve performance & availability.



5 INSTANCE LIFECYCLE

- Pending → Running → Stopping → Stopped → Terminated
- Reboot vs Stop/Start vs Terminate
- Use instance metadata service (IMDSv2).
- Handle lifecycle hooks in ASG for safe scale-in.
- Use Spot Instance interruptions notice (2 minutes).



6 CAPACITY PLANNING

- Understand workload patterns.
- Estimate baseline, peak & growth.
- Use CloudWatch metrics & Trusted Advisor.
- Consider:
  - ☑ Right sizing
  - ☑ Reserved / Savings Plans
  - ☑ Spot Instances
  - ☑ Multi-AZ / Multi-Region



7 PRODUCTION OPTIMIZATION

Performance
- ☑ Right instance family
- ☑ ENA enabled
- ☑ Placement groups
- ☑ EBS optimized

Availability
- ☑ Multi-AZ deployment
- ☑ Auto Scaling & ELB
- ☑ Health checks
- ☑ Failover strategies

Cost
- ☑ Right sizing
- ☑ Spot / Savings Plans
- ☑ Schedule instances
- ☑ Monitor & optimize

Security
- ☑ Least privilege IAM
- ☑ Security Groups
- ☑ Patch management
- ☑ IMDSv2 enforced

Observability
- ☑ CloudWatch metrics
- ☑ Logs & alarms
- ☑ X-Ray tracing
- ☑ Dashboards

SENIOR ENGINEER TIP

Optimize for the four pillars: Performance, Cost, Reliability & Security. Everything is a trade-off!

COMMON INTERVIEW FOCUS AREAS

- ☑ How do you choose the right instance?
- ☑ How do you handle sudden traffic spikes?
- ☑ How do you reduce EC2 costs?
- ☑ How do you ensure high availability?
- ☑ How do you handle instance failures?
- ☑ What is your capacity planning approach?

KEY METRICS TO MONITOR

- CPU Utilization
- Network In / Out
- Disk IOPS / Throughput
- Status Checks
- ASG Group In Service
- EBS Burst Balance

PAGE 03 OF 20

VERIQTA

@veriqta_

VPC & NETWORKING INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Design secure. Connect anything. Scale globally. Operate reliably.

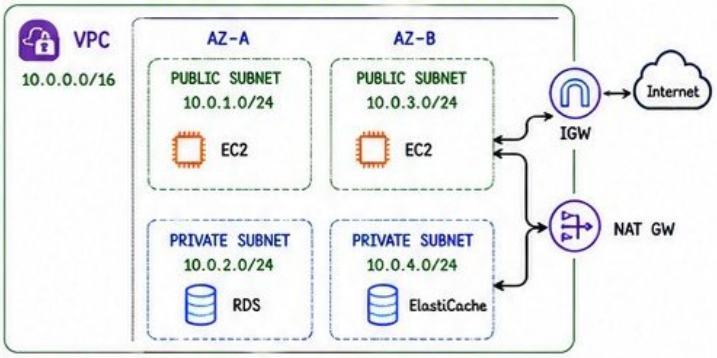
REMEMBER

- ☑ Network is the foundation
- ☑ Secure by default
- ☑ Isolate what can fail
- ☑ Observe everything
- ☑ Automate & document

TOPICS COVERED ON THIS PAGE

1. VPC Architecture
2. Public vs Private Subnets
3. Route Tables
4. Internet Gateway
5. NAT Gateway
6. Security Groups vs NACLs
7. Production Networking

TYPICAL PRODUCTION VPC ARCHITECTURE



KEY BENEFITS

- Network isolation
- Improved security
- High availability
- Scalability
- Cost optimization
- Full control

1 VPC ARCHITECTURE

- Your VPC is logically isolated from other AWS networks.
- You define your IP address range (CIDR block).
- Subnets split your VPC into smaller segments.
- Resources inside the VPC communicate using private IPs.
- You control routing, connectivity and security.



2 PUBLIC VS PRIVATE SUBNETS

PUBLIC SUBNET

- ✓ Has route to Internet Gateway (IGW)
- ✓ Resources get public IP (optional)
- ✓ Used for resources that need inbound access

Examples:

- Bastion Hosts
- ALB / NLB
- Public Web Servers

PRIVATE SUBNET

- ✓ No direct route to IGW
- ✓ Outbound via NAT Gateway (if needed)
- ✓ Used for backend resources

Examples:

- App Servers
- Databases (RDS)
- Caches, MQ, Workers

3 ROUTE TABLES

- A route table contains a set of rules (routes).
- Each subnet is associated with one route table.
- Longest prefix match is used.

DESTINATION	TARGET	DESCRIPTION
10.0.0.0/16	local	Local route (automatic)
0.0.0.0/0	igw-xxxxxxx	Internet access
0.0.0.0/0	nat-xxxxxxx	Outbound via NAT
pl-xxxxxxx	vgw-xxxxxxx	PrivateLink / VPN
10.1.0.0/16	pcx-xxxxxxx	Route to another VPC

4 INTERNET GATEWAY (IGW)

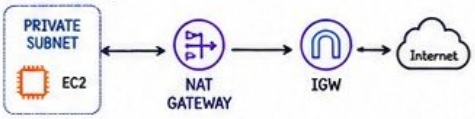
- A horizontally scaled, HA component.
- Attached to your VPC.
- Enables communication between your VPC and the Internet.



Important: IGW only works with IPv4 (for IPv6 use an egress-only IGW).

5 NAT GATEWAY

- Allows instances in private subnets to access the internet for outbound traffic.
- Does NOT allow inbound connections from the internet.
- Place NAT Gateway in a public subnet.
- One NAT Gateway per AZ for high availability.



Cost: Charged per hour + data processed.

6 SECURITY GROUPS vs NACLs

SECURITY GROUPS (SG)

- Stateful
- Allow rules only
- Applied at ENI level (instance)
- Default allow outbound
- Evaluate after NACL

NETWORK ACLS (NACL)

- Stateless
- Allow and Deny rules
- Applied at subnet level
- Default allow all
- Evaluate before SG

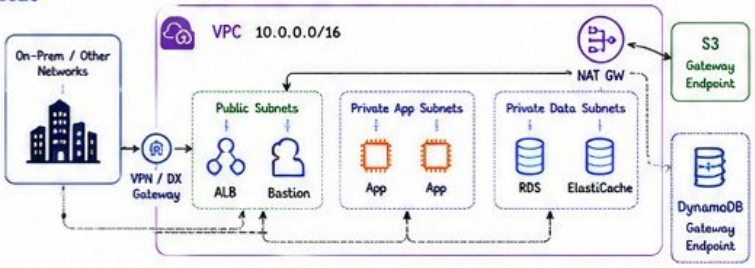
Type	Stateful	Stateless
Scope	Instance level	Subnet level
Rules	Allow only	Allow + Deny
Default	Allow all out	Allow all
Performance	Higher	Lower

Best Practice: Use SG for most cases. Use NACL for additional layer of defense or regulatory requirements.

7 PRODUCTION NETWORKING BEST PRACTICES

- ☑ Use multi-AZ architecture for high availability.
- ☑ Isolate workloads using private subnets.
- ☑ Use least privilege security rules.
- ☑ Centralize egress with NAT Gateway.
- ☑ Use VPC Endpoints (Gateway/Interface) to access AWS services privately.
- ☑ Enable Flow Logs and monitor traffic.
- ☑ Document your IP ranges and routing.
- ☑ Test failure scenarios regularly.

TYPICAL ENTERPRISE PRODUCTION NETWORK



Monitor with: CloudWatch • VPC Flow Logs • Route 53 • CloudTrail • AWS Config

SENIOR TIP

Great engineers design for failure, secure by default, and automate everything.

"Network is the invisible foundation of every reliable system."

PAGE 04 OF 20

VERIQTA

@veriqta_

LOAD BALANCING & TRAFFIC MANAGEMENT

SENIOR INTERVIEW TOPICS

Distribute smart. Route right. Keep users happy. Stay available.

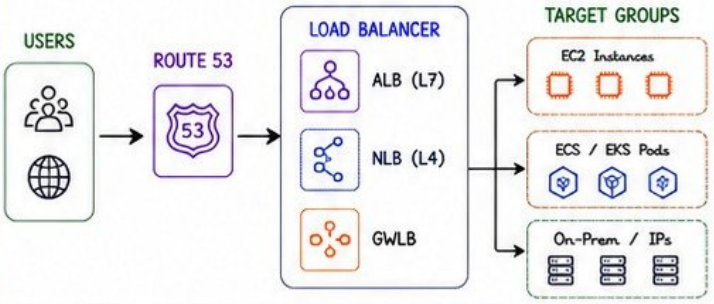
ENGINEER MINDSET

- ☑ Design for scale
- ☑ Optimize for performance
- ☑ Automate & observe
- ☑ Plan for failures
- ☑ Measure & improve

TOPICS COVERED ON THIS PAGE

1. Application Load Balancer (ALB)
2. Network Load Balancer (NLB)
3. Gateway Load Balancer (GWLB)
4. Target Groups
5. Health Checks
6. Session Persistence
7. Traffic Routing Strategies

WHERE LOAD BALANCERS FIT

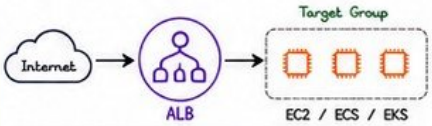


ALL ABOUT LB

- High availability
- Elastic scaling
- Fault tolerance
- Improved performance
- Centralized traffic control
- Secure & extensible

1 APPLICATION LOAD BALANCER (ALB)

- Operates at Layer 7 (Application Layer).
- HTTP/HTTPS traffic.
- Host-based & path-based routing.
- Supports microservices & containers.
- Ideal for web applications.

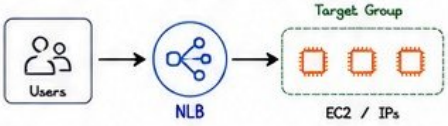


Use Cases

- ☑ Web apps
- ☑ Containers
- ☑ Microservices
- ☑ Multi-tenant apps

2 NETWORK LOAD BALANCER (NLB)

- Operates at Layer 4 (Transport Layer).
- TCP, UDP, TLS traffic.
- Ultra-high performance.
- Static IP support.
- Handles millions of requests per second.

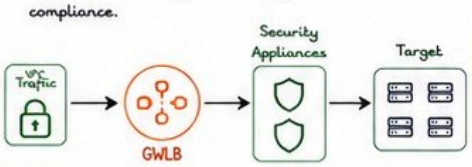


Use Cases

- ☑ High performance apps
- ☑ Gaming
- ☑ Real-time apps
- ☑ TCP/UDP services

3 GATEWAY LOAD BALANCER (GWLB)

- Operates at Layer 3.
- Transparently routes traffic to third-party virtual appliances.
- Useful for security, monitoring, compliance.



Use Cases

- ☑ Firewall inspection
- ☑ Threat detection
- ☑ IDS/IPS
- ☑ Network monitoring

4 TARGET GROUPS

- Group of targets (instances, IPs, pods, Lambdas).
- Registered with a Load Balancer.
- Traffic is distributed only to healthy targets.

Target Types

- ☑ Instance
- ☑ IP Address
- ☑ Lambda Function
- ☑ ALB (for chaining)



5 HEALTH CHECKS

- Monitor target health.
- Unhealthy targets are removed from rotation.
- Prevents traffic from going to bad instances.

Protocol	HTTP / HTTPS / TCP / gRRC
Path / Port	/health or custom path
Interval	10s (default)
Timeout	5s (default)
Healthy Threshold	2 (default)
Unhealthy Threshold	2 (default)

6 SESSION PERSISTENCE

- Ensures requests from the same client go to the same target.
- Improves user experience for stateful apps.

ALB
Cookie-based (ALB cookie)
Duration configurable

NLB
- Source IP-based
- Duration configurable

Use When

- ☑ User sessions are stateful
- ☑ Shopping carts
- ☑ Legacy applications

7 TRAFFIC ROUTING STRATEGIES

Round Robin (Default)
Distributes traffic evenly across targets.

Least Outstanding Requests
Sends traffic to target with least active requests.

Weighted Target Groups
Distribute traffic based on weights.

80%
20%

Path-Based Routing (ALB)
Routes based on URL path.

/api
/web

Host-Based Routing (ALB)
Routes based on domain name.

api.example.com
app.example.com

Canary / Blue-Green
Gradually shift traffic to new version.

90% → v1 (Blue)
10% → v2 (Green)

PRODUCTION BEST PRACTICES

- ☑ Use ALB for HTTP/HTTPS & microservices.
- ☑ Use NLB for performance & static IP needs.
- ☑ Use health checks meaningful to your app.
- ☑ Enable cross-zone load balancing when needed.
- ☑ Monitor metrics: 5XX, Latency, HealthyHostCount.
- ☑ Design for failure. Test regularly.

KEY METRICS TO WATCH

- Request Count
- Target Response Time
- HTTP 5XX Errors
- Healthy Host Count
- New Connections
- Active Connections

PAGE 05 OF 20

VERIQTA

@veriqta_

AWS STORAGE INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Store it right. Access it fast. Protect it always. Scale it endlessly.

ENGINEER MINDSET

- ☑ Design for durability
- ☑ Optimize for performance
- ☑ Automate backups
- ☑ Reduce costs smartly
- ☑ Test restores regularly

TOPICS COVERED ON THIS PAGE

1. Amazon S3
2. EBS (Elastic Block Store)
3. EFS (Elastic File System)
4. FSx (Fully Managed Storage)
5. Storage Lifecycle
6. Performance Optimization
7. Backup Strategies

AWS STORAGE OVERVIEW

S3	EBS	EFS	FSx
Object Storage	Block Storage	File Storage	Managed File Storage

CHOOSE THE RIGHT STORAGE

- S3 → Object storage for unstructured data, backups, logs, static content.
- EBS → Block storage for EC2 instances (OS, databases, applications).
- EFS → Shared file storage for Linux workloads (many-to-many access).
- FSx → Fully managed file systems (Windows, Lustre, ONTAP, OpenZFS).

KEY BENEFITS

- Durability & reliability
- Virtually unlimited scale
- High performance options
- Cost optimization
- Secure & encrypted
- Highly available
- Automated lifecycle

1 AMAZON S3

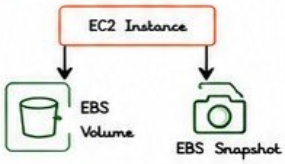
- Object storage built for 99.999999999% (11 9's) durability.
- Stores data as objects in buckets.
- Ideal for backups, logs, media, data lakes, static websites.
- Features: Versioning, Lifecycle, Replication, SSE-KMS, CRR, CORS.

S3 STORAGE CLASSES

- Standard – Frequent access
- IA – Infrequent access
- Glacier Instant Retrieval – Archive
- Glacier Flexible Retrieval – Long-term
- Glacier Deep Archive – Lowest cost

EBS (ELASTIC BLOCK STORE)

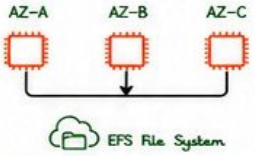
- Block storage for EC2 instances.
- Attached to a single AZ.
- Use cases: OS, databases, applications, caches.
- Volume types:
  - gp3 (General Purpose SSD)
  - io2 / io2 Block Express (High IOPS)
  - st1 (Throughput Optimized HDD)
  - sc1 (Cold HDD)
  - Magnetic (Legacy)



EFS (ELASTIC FILE SYSTEM)

- Fully managed NFS file system.
- Accessible from many EC2 instances across AZs.
- Use cases: Web content, media processing, shared home directories.
- Features: Automatic scaling, high availability, lifecycle management.

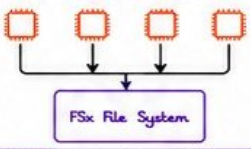
MULTI-AZ ACCESS



FSx (FULLY MANAGED STORAGE)

- Managed file systems for specific workloads.
- Options:
  - FSx for Windows File Server
  - FSx for Lustre (High Performance)
  - FSx for NetApp ONTAP
  - FSx for OpenZFS
  - FSx for OpenZFS (S3-compatible)

APPLICATIONS / CLIENTS



5 STORAGE LIFECYCLE

- Automatically move data to the most cost-effective tier.

S3 LIFECYCLE EXAMPLE



☑ Lifecycle policies reduce costs without affecting availability.

6 PERFORMANCE OPTIMIZATION

S3 OPTIMIZATION

- ☑ Use appropriate storage class
- ☑ Enable S3 Transfer Acceleration
- ☑ Use CloudFront for global delivery
- ☑ Multipart upload for large files
- ☑ Enable compression
- ☑ Use VPC Endpoints

EBS OPTIMIZATION

- ☑ Use gp3 with right IOPS/Throughput
- ☑ Enable EBS Multi-Attach (io1/io2)
- ☑ Provision IOPS for consistent perf.
- ☑ Use RAID 0 for max throughput
- ☑ Place in same AZ as EC2
- ☑ Monitor with CloudWatch

7 BACKUP STRATEGIES

- Follow 3-2-1 Backup Rule
- Automate & verify backups
- Encrypt backups
- Test restore regularly
- Document recovery process

SERVICE	BACKUP OPTIONS	BEST PRACTICES
S3	• Versioning • Cross-Region Replication (CRR) • AWS Backup for S3	• Enable versioning • Replicate to another region • Use lifecycle to protect old versions
EBS	• EBS Snapshots • AWS Backup (EBS) • AMIs for long-term retention	• Schedule regular snapshots • Copy snapshots to another region • Delete old snapshots (policy)
EFS	• AWS Backup for EFS • EFS to EFS replication (same region)	• Automate backups • Store backups in separate account
FSx	• FSx Backups (automatic) • Snapshots to S3 (where supported)	• Enable automatic backups • Replicate to DR region

INTERVIEW QUICK TIPS

- Understand the data type and access pattern.
- Choose the right storage service.
- Design for durability, availability and cost.
- Always plan for backup and recovery.

PRO TIP

Great engineers design storage that is secure, cost-effective, highly available and easy to restore.

REMEMBER

Backups are useless until restores are tested.

GOLDEN RULE

The right storage choice today prevents outages tomorrow.

PAGE 06 OF 20

VERIQTA

@veriqta_

AWS DATABASE ARCHITECTURE

INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Design for scale. Ensure availability. Protect data. Optimize performance.

ENGINEER MINDSET

- ☑ Think in systems, not services
- ☑ Design for failure
- ☑ Automate everything
- ☑ Measure & optimize
- ☑ Security by design

TOPICS COVERED ON THIS PAGE

1. Amazon RDS
2. Aurora (MySQL / PostgreSQL)
3. DynamoDB
4. ElastiCache (Redis / Memcached)
5. Multi-AZ Deployment
6. Read Replicas
7. Database Scalability

AWS DATABASE SERVICES OVERVIEW

Amazon RDS	Amazon Aurora	Amazon DynamoDB	Amazon ElastiCache
Relational Managed DB	High performance Relational DB	NoSQL Key-Value / Document DB	In-Memory Cache Service

CHOOSE THE RIGHT DATABASE

RDS / Aurora	→	Relational workloads (OLTP), ACID, joins, transactions
DynamoDB	→	Serverless, massive scale, low latency, key-value/document
ElastiCache	→	Caching layer, sessions, leaderboards, queues, performance boost

WHY IT MATTERS

- High availability & durability
- Performance at scale
- Cost optimization
- Low latency
- Security & compliance
- Automated scaling
- Easy backup & recovery

1 AMAZON RDS

- Managed relational databases.
- Supports: MySQL, PostgreSQL, MariaDB, Oracle, SQL Server.
- Automated backups, patching, monitoring, failover.
- Use cases: Web apps, ERP, CRM, internal applications.

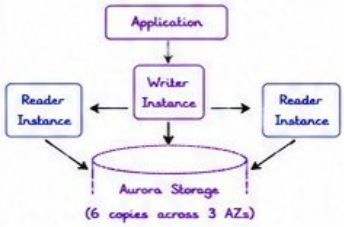
RDS ARCHITECTURE (SINGLE-AZ)



2 AURORA (MYSQL / POSTGRESQL)

- AWS-built, MySQL & PostgreSQL compatible.
- Up to 5x performance vs MySQL.
- Storage auto-scales in 10GB increments.
- Highly available & durable (6 copies across 3 AZs).

AURORA ARCHITECTURE (MULTI-AZ)



3 DYNAMODB

- Fully managed NoSQL database.
- Key-Value / Document database.
- Single-digit millisecond latency at any scale.
- Automatic scaling (On-Demand or Provisioned).
- Use cases: User profiles, carts, IoT, payments, mobile apps.

DYNAMODB TABLE

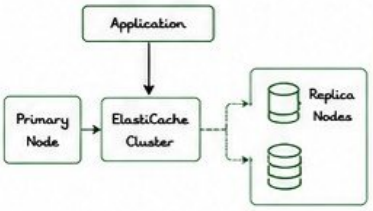
Partition Key	Sort Key (Optional)
UserID	Timestamp
Item	Attributes (JSON)

Scales automatically
No servers to manage!

4 ELASTICACHE (REDIS / MEMCACHED)

- In-memory caching service.
- Supports Redis (advanced features) & Memcached.
- Sub-millisecond latency.
- Use cases: Session store, caching, leaderboards, rate limiting.

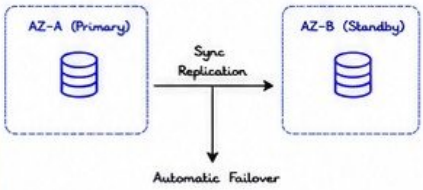
ELASTICACHE ARCHITECTURE



5 MULTI-AZ DEPLOYMENT

- Synchronous replication to a standby in different AZ.
- Automatic failover in case of AZ failure.
- No data loss for RDS (synchronous).

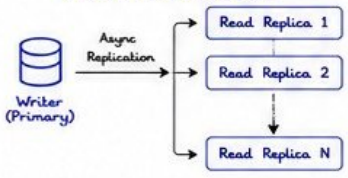
MULTI-AZ DIAGRAM (RDS / AURORA)



6 READ REPLICAS

- Asynchronous replication from source to replicas.
- Offload read traffic & improve performance.
- Use for reporting, analytics, read-heavy workloads.
- Aurora allows up to 15 read replicas.

READ REPLICA ARCHITECTURE



7 DATABASE SCALABILITY

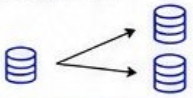
VERTICAL SCALING (Scale Up)

- Increase instance size (CPU, RAM).
- Limited by instance limits.
- Downtime may be required.



HORIZONTAL SCALING (Scale Out)

- Read replicas, sharding, partitioning.
- Used for high throughput & massive scale.



ELASTIC SCALING STRATEGIES

- ☑ Auto Scaling Groups (for EC2 read replicas)
- ☑ Cache scaling (ElastiCache clusters)
- ☑ On-Demand capacity (DynamoDB, Aurora Serverless)
- ☑ Storage auto-scaling (Aurora, DynamoDB)

QUICK COMPARISON

Service	Type	Best For	Scaling	Latency	Managed
RDS	Relational	Traditional apps (OLTP)	Vertical	Low	✓
Aurora	Relational	High performance apps	Vertical + Reader Scaling	Very Low	✓
DynamoDB	NoSQL	Massive scale, Serverless apps	Auto (On-Demand / Provisioned)	Single-digit ms	✓
ElastiCache	In-Memory	Caching, Sessions, Queues	Vertical + Cluster Mode	Sub-ms	✓

BACKUP STRATEGIES

- Automated Backups (RDS / Aurora) → Point-in-time recovery
- Snapshots (manual / automated) → Cross-region copies
- DynamoDB PITR (Point-in-time recovery) → Up to 35 days
- Cross-Region Replication → Disaster recovery

INTERVIEW QUICK TIPS

- Always consider availability, durability, and performance first.
- Explain trade-offs between cost, performance, and consistency.
- Think in terms of workloads: read-heavy, write-heavy, or balanced.
- Mention monitoring, backups, and security in your answer.

PAGE 07 OF 20

VERIQTA

@veriqta_

IDENTITY & SECURITY INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Secure by design. Least privilege always. Protect identities. Protect everything.

ENGINEER MINDSET

- ☑ Assume breach
- ☑ Secure identities
- ☑ Least privilege
- ☑ Automate security
- ☑ Monitor & improve

TOPICS COVERED ON THIS PAGE

1. IAM Architecture
2. Roles and Policies
3. Cross-Account Access
4. AWS Organizations
5. KMS (Key Management Service)
6. Secrets Manager
7. Least Privilege Design

AWS IDENTITY & SECURITY OVERVIEW



Secure identities → Control access → Encrypt data → Manage secrets → Monitor everything

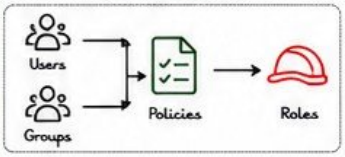
KEY BENEFITS

- Strong access control
- Centralized governance
- Data protection
- Secret management
- Compliance ready
- Audit & monitoring
- Reduced risk & cost

1 IAM ARCHITECTURE

- IAM is a global service.
- Root user has full access.
- IAM users are people.
- IAM roles are for applications.
- Permissions are defined in policies.
- Policies are attached to users, groups, or roles.

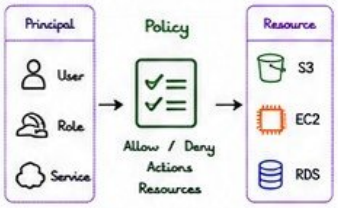
IAM ARCHITECTURE



Best Practice: Never use the root user for daily tasks.

2 ROLES & POLICIES

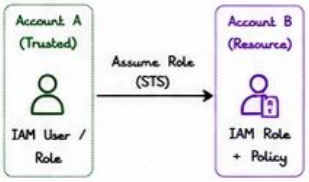
- Roles for applications and cross-account access.
- Policies define permissions.
- Use AWS managed policies when possible.
- Create customer managed policies for specific needs.



Best Practice: Attach policies to roles, not users.

3 CROSS-ACCOUNT ACCESS

- Use IAM roles to access resources in another AWS account.
- Avoid long-term access keys.
- Use External ID for third-party access.
- Enforce least privilege.

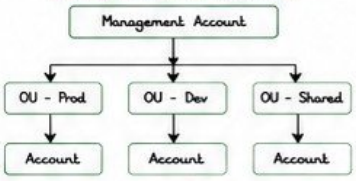


Best Practice: Use roles + STS, never share credentials.

4 AWS ORGANIZATIONS

- Manage multiple AWS accounts centrally.
- Organize accounts using OUs.
- Apply Service Control Policies (SCPs).
- Enable centralized billing, security & compliance.
- Use consolidated logging & auditing.

AWS ORGANIZATIONS STRUCTURE

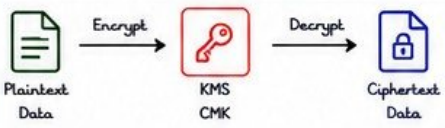


Best Practice: Use SCPs to set guardrails, not to manage permissions.

5 KMS (KEY MANAGEMENT SERVICE)

- KMS is used to create and manage encryption keys.
- Customer Managed Keys (CMK) give more control.
- Used for EBS, S3, RDS, Secrets Manager, Lambda, etc.
- Supports envelope encryption.
- Use key policies & IAM policies together.

KMS ENCRYPTION FLOW



Best Practice: Rotate keys, enable key deletion protection, and audit key usage.

6 SECRETS MANAGER

- Store, rotate, and manage secrets securely.
- Supports automatic secret rotation.
- Integrates with RDS, Redshift, DocumentDB, and more.
- Use resource-based policies for fine-grained access.

SECRETS MANAGER WORKFLOW



Best Practice: Never hardcode secrets. Use IAM roles + Secrets Manager.

7 LEAST PRIVILEGE DESIGN

- Grant only the minimum permissions required.
- Start with deny by default.
- Use condition keys for fine-grained control.
- Regularly review and audit permissions.
- Remove unused access.

LEAST PRIVILEGE PRINCIPLES

- Identify the exact need
- Grant minimum permissions
- Use policies with conditions
- Monitor & review access
- Remove unnecessary access

EXAMPLE CONDITION

```
{
  "Condition": {
    "StringEquals": {
      "aws:RequestedRegion":
        ["us-east-1"]
    }
  }
}
```

Golden Rule: Least privilege is not a feature, it is a mindset.

COMMON IAM ENTITIES

Entity	Description	Use Case
User	Represents a person	Console access
Group	Collection of users	Permissions management
Role	Assumed by users or services	Applications & cross-account
Policy	JSON document of permissions	Define access rules
Permission Boundary	Limits max permissions	Prevent privilege escalation
Session Policy	Temporary permissions	STS temporary access

EXAMPLE POLICY (S3 READ ONLY)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": ["arn:aws:s3:::example-bucket"]
    },
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": ["arn:aws:s3:::example-bucket/*"]
    }
  ]
}
```

SECURITY BEST PRACTICES CHECKLIST

- ☑ Use IAM roles instead of long-term keys
- ☑ Enable MFA for all privileged users
- ☑ Apply least privilege to all identities
- ☑ Use AWS Organizations & SCPs for guardrails
- ☑ Encrypt data at rest and in transit
- ☑ Store secrets in Secrets Manager
- ☑ Enable CloudTrail, GuardDuty, and Security Hub
- ☑ Regularly review access with IAM Access Analyzer
- ☑ Automate security with IaC and CI/CD

PAGE 08 OF 20

VERIQTA

@veriqta_

SERVERLESS & EVENT-DRIVEN ARCHITECTURE INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Build less. Scale more. Automate everything. Pay for value.

ENGINEER MINDSET

- ☑ Design for failure
- ☑ Decouple everything
- ☑ Idempotent by default
- ☑ Observe & automate
- ☑ Optimize cost & latency

TOPICS COVERED ON THIS PAGE

1. AWS Lambda
2. API Gateway
3. Amazon EventBridge
4. Amazon SNS
5. Amazon SQS
6. AWS Step Functions
7. Production Serverless Design

SERVERLESS & EVENT-DRIVEN OVERVIEW



Events → Route → Compute → Integrate → Automate → Scale

WHY SERVERLESS?

- No server management
- Automatic scaling
- High availability by default
- Pay per request/usage
- Faster development
- Focus on business logic

1 AWS LAMBDA

- Run code without provisioning servers.
- Supports multiple runtimes.
- Auto scales from 0 to thousands.
- Pay per request + duration.
- Integrates with all AWS services.

COMMON TRIGGERS

API Gateway, S3, DynamoDB, EventBridge, SNS / SQS

BEST PRACTICES

- ☑ Keep functions small & focused
- ☑ Use environment variables
- ☑ Enable X-Ray tracing
- ☑ Set appropriate memory & timeout
- ☑ Handle errors & retries properly

2 API GATEWAY

- Create, publish, secure & monitor APIs.
- Supports REST, HTTP & WebSocket APIs.
- Integrates with Lambda & other services.
- Handles auth, throttling, caching.
- High availability & global scale.

API TYPES

REST API (v1)
Full features, broader support

HTTP API (v2)
Lower latency, higher performance

WebSocket API
Real-time, bidirectional

BEST PRACTICES

- ☑ Use IAM or Cognito for auth
- ☑ Enable throttling & rate limiting
- ☑ Cache responses for performance
- ☑ Enable logging & metrics
- ☑ Use stages & custom domains

3 EVENTBRIDGE

- Serverless event bus.
- Route events from AWS services, SaaS apps, or custom apps.
- Event-driven architecture at scale.
- Supports schema registry & archive.
- Great for decoupling services.

EVENT SOURCES

- AWS Services
- SaaS Applications
- Custom Applications
- Scheduled Events (Cron)

BEST PRACTICES

- ☑ Use event patterns to filter events
- ☑ Archive important events
- ☑ Use DLQ for failed targets
- ☑ Monitor with CloudWatch
- ☑ Keep events small & meaningful

4 AMAZON SNS

- Fully managed pub/sub messaging.
- Fan-out notifications to many endpoints.
- Supports HTTP/S, Email, SMS, Lambda, SQS, Mobile Push.
- High throughput & durability.

USE CASES

Alerts, User Notifications, Mobile Push, Email Delivery

BEST PRACTICES

- ☑ Use message filtering
- ☑ Use FIFO topics for ordering
- ☑ Encrypt with KMS
- ☑ Monitor delivery status
- ☑ Use subscriptions with DLQ

5 AMAZON SQS

- Decouple & buffer messages between distributed systems.
- Standard (at-least-once) & FIFO (exactly-once + order).
- Scales automatically.
- Retains messages until processed.

SQS TYPES

Standard Queue
- High throughput
- Best-effort order

FIFO Queue
- Exactly-once
- Ordered
- High consistency

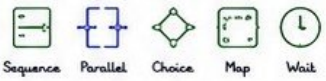
BEST PRACTICES

- ☑ Set appropriate visibility timeout
- ☑ Use DLQ for poison messages
- ☑ Batch messages when possible
- ☑ Enable encryption (SSE-KMS)
- ☑ Monitor depth & age of queue

6 AWS STEP FUNCTIONS

- Visual workflows for serverless apps.
- Orchestrate Lambda, APIs, SQS, SNS, DynamoDB & more.
- Built-in error handling, retries & timeouts.
- Supports parallel, map, wait, choice states.

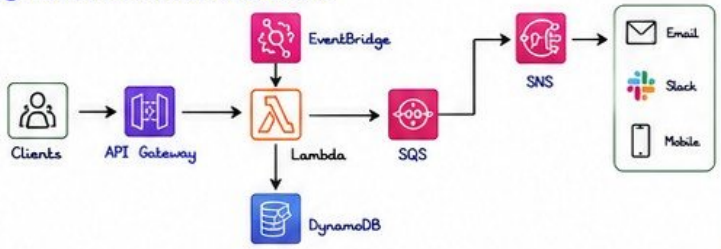
WORKFLOW PATTERNS



BEST PRACTICES

- ☑ Keep workflows simple & modular
- ☑ Use timeouts & retries wisely
- ☑ Store state in execution input/output
- ☑ Use Express Workflows for high scale
- ☑ Enable logging & X-Ray tracing

7 PRODUCTION SERVERLESS DESIGN



DESIGN PRINCIPLES

- ☑ Decouple services with events & queues
- ☑ Idempotent functions
- ☑ Fail fast & handle errors gracefully
- ☑ Use least privilege IAM roles
- ☑ Observe everything (logs, metrics, traces)
- ☑ Optimize retries, timeouts & concurrency
- ☑ Secure by default (encrypt, validate, auth)
- ☑ Automate deployments & monitoring

EVENT-DRIVEN FLOW EXAMPLE



SERVERLESS OBSERVABILITY

- CloudWatch Logs — Centralized logs
- AWS X-Ray — Distributed tracing
- CloudWatch Metrics — Performance metrics
- Alarms — Proactive alerting

INTERVIEW QUICK TIPS

- Understand event flow and service integrations.
- Know when to use each service.
- Explain scalability, reliability & cost.
- Discuss monitoring, security & best practices.
- Think in terms of decoupling & automation.

PAGE 09 OF 20

VERIQTA

@veriqta_

AWS CONTAINERS & KUBERNETES

INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Run anywhere. Scale intelligently. Automate relentlessly.

ENGINEER MINDSET

- ☑ Automate everything
- ☑ Design for resiliency
- ☑ Optimize for scale
- ☑ Observe & improve
- ☑ Security by design

TOPICS COVERED ON THIS PAGE

1. Amazon ECS
2. Amazon EKS
3. AWS Fargate
4. Container Deployment
5. Cluster Architecture
6. Service Discovery
7. Production Orchestration

AWS CONTAINER SERVICES OVERVIEW



Build once → Push to ECR → Deploy on ECS / EKS / Fargate → Scale
Monitor → Secure → Optimize

WHY IT MATTERS

- Faster deployments
- High availability
- Cost optimization
- Scalable on demand
- Portability & flexibility
- Infrastructure automation
- Better resource utilization

1 AMAZON ECS

- Fully managed container orchestration.
- Supports EC2 launch type and Fargate.
- Tight integration with AWS services.
- Ideal for teams that want simplicity.
- Supports Service Auto Scaling.

ECS COMPONENTS

- Cluster
- Task Definition
- Service
- Tasks (Containers)
- Capacity (EC2 / Fargate)

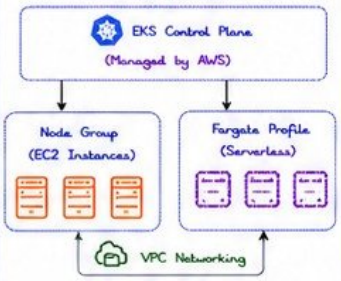
BEST PRACTICES

- ☑ Use Fargate for serverless operations.
- ☑ Use EC2 for specialized workloads.
- ☑ Keep task definitions immutable.
- ☑ Enable Container Insights.
- ☑ Use least privilege IAM roles.

2 AMAZON EKS

- Managed Kubernetes control plane.
- Runs Kubernetes natively on AWS.
- Support for EC2 and Fargate nodes.
- Community tools and ecosystem.
- Highly customizable and flexible.

EKS ARCHITECTURE



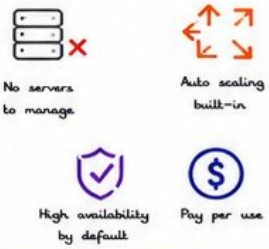
BEST PRACTICES

- ☑ Use managed node groups.
- ☑ Keep Kubernetes versions updated.
- ☑ Enable IRSA for fine-grained access.
- ☑ Use Fargate for bursty workloads.
- ☑ Monitor with Container Insights & Prometheus.

3 AWS FARGATE

- Serverless compute engine for containers.
- No server management.
- Pay for vCPU & memory used.
- Ideal for microservices, batch jobs.
- Works with ECS and EKS.

FARGATE BENEFITS



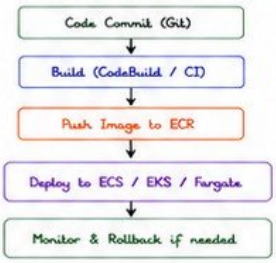
BEST PRACTICES

- ☑ Use appropriate CPU & memory.
- ☑ Design stateless containers.
- ☑ Use Fargate Spot for cost savings.
- ☑ Enable EFS for persistent storage.

4 CONTAINER DEPLOYMENT

- Use CI/CD pipelines for automation.
- Store images in Amazon ECR.
- Blue/Green or Rolling deployments.
- Health checks & rollback strategy.
- Version your images and configs.

DEPLOYMENT FLOW



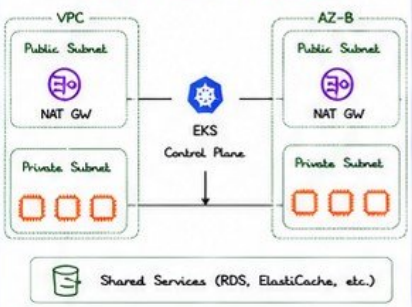
BEST PRACTICES

- ☑ Scan images for vulnerabilities.
- ☑ Use immutable infrastructure.
- ☑ Automate rollbacks.
- ☑ Use canary for critical releases.

5 CLUSTER ARCHITECTURE

- Design for availability, scalability & security.
- Multi-AZ nodes for high availability.
- Separate workloads using namespaces / services.
- Use Auto Scaling for node groups.

EKS CLUSTER ARCHITECTURE (EXAMPLE)



6 SERVICE DISCOVERY

- Services must find each other reliably.
- Kubernetes DNS in EKS.
- AWS Cloud Map for hybrid workloads.
- Health-aware service discovery.

SERVICE DISCOVERY OPTIONS

Kubernetes DNS

- CoreDNS
- ClusterIP services
- Internal resolution
- Pod & Service discovery

AWS Cloud Map

- Private DNS namespaces
- Works with ECS & EC2
- Health checks
- Cross-VPC discovery

BEST PRACTICES

- ☑ Use meaningful service names.
- ☑ Enable health checks.
- ☑ Use namespaces to isolate apps.

7 PRODUCTION ORCHESTRATION

- Orchestrate for reliability and scale.
- Use autoscaling for demand changes.
- Use secrets management for sensitive data.
- Observability and logging are critical.
- Disaster recovery & backup strategy.

ORCHESTRATION BEST PRACTICES

- ☑ Enable Horizontal Pod Autoscaler (HPA).
- ☑ Use Cluster Autoscaler for nodes.
- ☑ Use AWS Secrets Manager / SSM.
- ☑ Centralize logs (CloudWatch / Loki / ELK).
- ☑ Set resource requests and limits.
- ☑ Implement pod disruption budgets (PDB).
- ☑ Test failover and recovery regularly.

COMMON TOOLS & INTEGRATIONS

- Amazon ECR – Container Registry
- AWS CodePipeline – CI/CD
- AWS CodeBuild – Build Images
- AWS CloudWatch – Monitoring
- AWS X-Ray – Distributed Tracing
- AWS ALB / NLB – Traffic Management
- AWS Secrets Manager – Secrets
- Amazon EFS / EBS – Storage
- AWS IAM / IRSA – Access Control

PRO TIP
Containers are not just about running apps, they are about running production reliably.

REMEMBER
Great container engineers design for resilience, observability, and scale.

GOLDEN RULE
Automate everything. Manual processes do not scale.

PAGE 10 OF 20

VERIQTA

@veriqta_

MONITORING & OBSERVABILITY INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

You can't improve what you can't see. Monitor everything. Alert intelligently. Act fast.

ENGINEER MINDSET

- ☑ Observe everything
- ☑ Detect early
- ☑ Alert smartly
- ☑ Automate response
- ☑ Review & improve

TOPICS COVERED ON THIS PAGE

1. Amazon CloudWatch
2. AWS CloudTrail
3. AWS Config
4. AWS X-Ray
5. Centralized Logging
6. Alerting Strategy
7. Operational Dashboards

AWS OBSERVABILITY OVERVIEW



Observe → Understand → Alert → Automate → Improve

WHY IT MATTERS

- Faster incident detection
- Better troubleshooting
- Audit & compliance
- Proactive operations
- Cost optimization
- Business reliability
- Data-driven decisions

1 AMAZON CLOUDWATCH

- Native monitoring & observability service.
- Metrics, Logs, Alarms, Dashboards.
- High resolution metrics.
- Automated actions.
- Custom metrics & namespaces.

KEY COMPONENTS



BEST PRACTICES

- ☑ Use meaningful metric names
- ☑ Set appropriate alarm thresholds
- ☑ Use anomaly detection
- ☑ Retain logs with lifecycle policies
- ☑ Review dashboards regularly

2 AWS CLOUDTRAIL

- Tracks API activity & account events.
- Provides governance, compliance, and auditing.
- Delivers logs to S3.
- Integrates with CloudWatch Logs & EventBridge.

EVENT TYPES

Management Events	Data Events
• Console actions	• S3 object events
• API operations	• Lambda invokes
• IAM changes	• DynamoDB events
• Resource changes	• Read/Write activity

BEST PRACTICES

- ☑ Enable multi-region trails
- ☑ Store logs in dedicated S3 bucket
- ☑ Enable log file validation
- ☑ Monitor CloudTrail with alarms

3 AWS CONFIG

- Records configuration changes of AWS resources.
- Evaluates resources against rules.
- Tracks compliance over time.
- Supports remediation.

CORE FEATURES



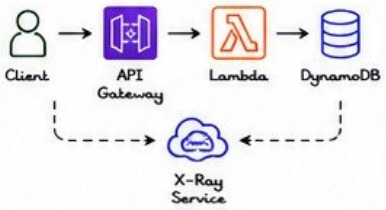
BEST PRACTICES

- ☑ Enable in all regions
- ☑ Use managed & custom rules
- ☑ Automate remediation
- ☑ Review compliance dashboards

4 AWS X-RAY

- Distributed tracing service.
- Analyzes request flow end-to-end.
- Identifies latency & errors.
- Works with Lambda, ECS, API GW, EC2, ALB, SDKs.

TRACE FLOW



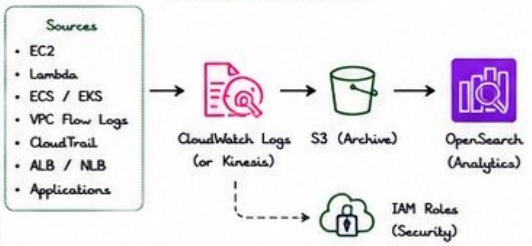
BEST PRACTICES

- ☑ Instrument all services
- ☑ Use annotations & metadata
- ☑ Analyze service maps
- ☑ Trace slow & failed requests

5 CENTRALIZED LOGGING

- Centralize logs from all accounts & services.
- Use CloudWatch Logs, S3, OpenSearch, or third-party tools.
- Enables search, analysis & long-term retention.
- Foundation for observability & security.

COMMON ARCHITECTURE



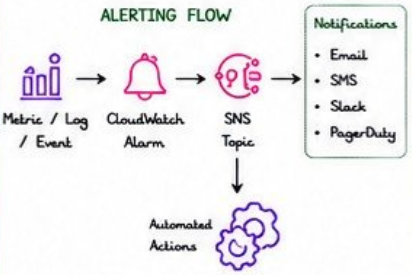
BEST PRACTICES

- ☑ Use structured logging (JSON)
- ☑ Standardize log formats
- ☑ Encrypt logs at rest & in transit
- ☑ Set retention & lifecycle policies
- ☑ Use log subscriptions for analytics
- ☑ Monitor log ingestion & errors

6 ALERTING STRATEGY

- Alert early. Alert smart. Reduce noise.
- Use CloudWatch Alarms, EventBridge, SNS, and Incident Manager.
- Escalate based on severity.

ALERTING FLOW



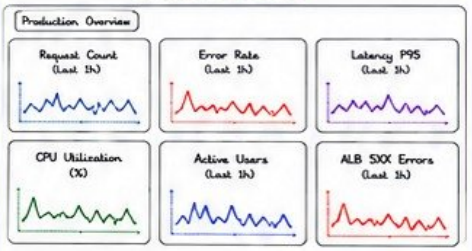
BEST PRACTICES

- ☑ Use appropriate thresholds
- ☑ Avoid alert fatigue
- ☑ Group related alarms
- ☑ Test alerts regularly
- ☑ Use composite alarms
- ☑ Route by severity
- ☑ Automate runbook actions

7 OPERATIONAL DASHBOARDS

- Dashboards provide real-time visibility.
- Track health, performance, and business KPIs.
- Build for operators and stakeholders.

DASHBOARD EXAMPLE



BEST PRACTICES

- ☑ Keep it simple & focused
- ☑ Show trends, not just numbers
- ☑ Use meaningful metrics
- ☑ Add drill-down links & filters
- ☑ Refresh automatically
- ☑ Separate dashboards for teams

PRO TIP

Observability is not a tool. It's a practice. Instrument everything, correlate data, and act fast.

REMEMBER

Good observability reduces MTTR and improves reliability.

GOLDEN RULE

If it's not monitored, it doesn't exist. If it doesn't alert, it doesn't matter.

PAGE 11 OF 20

VERIQTA

@veriqta_

CI/CD ON AWS INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Automate everything. Deliver faster. Deploy safer. Build better.

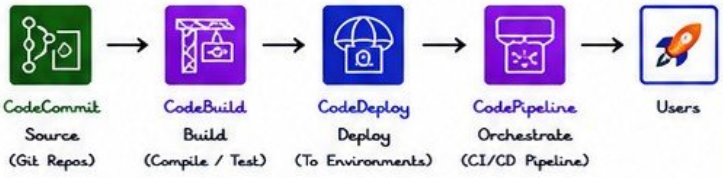
ENGINEER MINDSET

- ☑ Automate with purpose
- ☑ Ship small, ship safe
- ☑ Test early, test often
- ☑ Secure every pipeline
- ☑ Measure & improve

TOPICS COVERED ON THIS PAGE

1. AWS CodeCommit
2. AWS CodeBuild
3. AWS CodeDeploy
4. AWS CodePipeline
5. Blue/Green Deployment
6. Canary Deployment
7. Pipeline Security

AWS CI/CD OVERVIEW



Commit → Build → Test → Deploy → Monitor → Improve

WHY IT MATTERS

- Faster delivery of features
- High quality releases
- Repeatable & reliable
- Safe & automated rollouts
- Lower risk of failure
- Better developer productivity
- End-to-end visibility

1 AWS CODECOMMIT

- Fully managed Git-based source control.
- Hosts private repositories.
- Scales with IAM integration.
- Works with any Git client.
- Secure & highly available.

KEY FEATURES

- Managed Repositories
- Pull Requests
- Branch Permissions
- Encryption (KMS)
- Fine-grained IAM

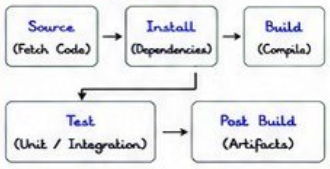
BEST PRACTICES

- ☑ Use meaningful repo structure.
- ☑ Enable branch protection rules.
- ☑ Integrate with CodePipeline.
- ☑ Scan commits with security tools.

2 AWS CODEBUILD

- Fully managed build service.
- Compiles source code.
- Runs unit tests.
- Generates build artifacts.
- Scales automatically.

BUILD PHASES



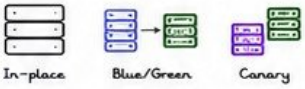
BEST PRACTICES

- ☑ Use buildspec.yml.
- ☑ Cache dependencies.
- ☑ Run tests in parallel.
- ☑ Store artifacts in S3.
- ☑ Use environment variables.

3 AWS CODEDEPLOY

- Deploys applications to compute services.
- Supports EC2, ECS, Lambda.
- Automates deployments.
- Monitors deployment health.
- Handles rollback automatically.

DEPLOYMENT TYPES



LIFECYCLE EVENTS



BEST PRACTICES

- ☑ Use health checks & alarms.
- ☑ Automate rollback on failure.
- ☑ Keep deployments idempotent.
- ☑ Store logs for every deployment.

4 AWS CODEPIPELINE

- Orchestrates your CI/CD workflow.
- Model workflows as a pipeline.
- Integrates AWS services & 3rd parties.
- Visualize & automate delivery.
- Release changes with confidence.

PIPELINE STAGES (EXAMPLE)



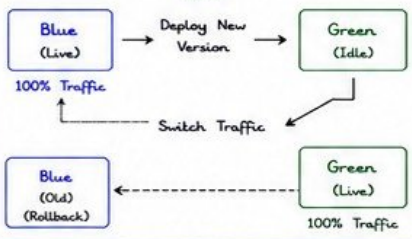
BEST PRACTICES

- ☑ Keep stages small & focused.
- ☑ Use manual approvals for prod.
- ☑ Enable notifications.
- ☑ Version-control your pipeline.

5 BLUE/GREEN DEPLOYMENT

- Two identical environments.
- One (Blue) serves traffic.
- Other (Green) is new version.
- Instant switch with zero downtime.
- Easy rollback by switching back.

FLOW

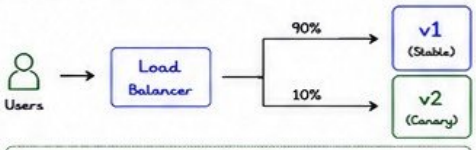


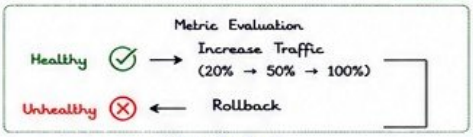
BEST PRACTICES

- ☑ Use ELB / ALB / Route53 for switch.
- ☑ Automate health checks before cutover.
- ☑ Keep environments identical.

6 CANARY DEPLOYMENT

- New version released to small % of users.
- Validate with real traffic.
- Gradually increase traffic.
- Rollback quickly if errors detected.
- Lower risk than all-at-once.





BEST PRACTICES

- ☑ Use CloudWatch alarms for metrics.
- ☑ Define SLOs before rollout.
- ☑ Automate promotion or rollback.

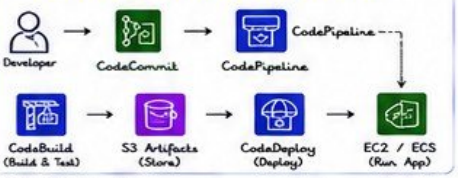
7 INTPELINE SECURITY

- Secure the pipeline, not just the app.
- Protect source, builds, artifacts, deployments.
- Follow least privilege principle.
- Detect & prevent threats early.
- Audit everything.

SECURITY CHECKLIST

- ☑ Use IAM roles, not long-term keys.
- ☑ Encrypt artifacts in S3 (SSE-KMS).
- ☑ Enable CodeBuild privileged mode only if needed.
- ☑ Scan code (SAST) & dependencies.
- ☑ Scan containers (ECR vulnerability scanning).
- ☑ Sign & verify artifacts.
- ☑ Use approvals for production.
- ☑ Log & monitor every action (CloudTrail).
- ☑ Restrict network access (VPC, SG, NACL).
- ☑ Rotate secrets (Secrets Manager / Parameter Store).

CI/CD PIPELINE EXAMPLE ARCHITECTURE



DEPLOYMENT STRATEGY COMPARISON

Strategy	Downtime	Risk	Speed	Rollback	Best For
In-place	High	High	Fast	Hard	Small changes
Blue/Green	None	Low	Medium	Easy	Critical apps
Canary	None	Very Low	Medium	Easy	Continuous delivery

INTERVIEW QUICK TIPS

- Explain each service in 1–2 lines.
- Describe end-to-end pipeline flow.
- Compare Blue/Green vs Canary.
- How do you rollback?
- How do you secure the pipeline?
- Which metrics do you monitor?

PAGE 12 OF 20

VERIQTA

@veriqta_

INFRASTRUCTURE AS CODE (IaC)

INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Code your infrastructure. Version it. Test it. Deploy it. Trust it.

ENGINEER MINDSET

- ☑ Automate everything
- ☑ Eliminate manual steps
- ☑ Make it repeatable
- ☑ Detect drift early
- ☑ Design for scale
- ☑ Continuously improve

TOPICS COVERED ON THIS PAGE

1. AWS CloudFormation
2. AWS CDK
3. Terraform on AWS
4. State Management
5. Drift Detection
6. Module Design
7. Enterprise Automation

IaC OVERVIEW & BENEFITS



Write Code	Version Control	Review & Test	Deploy	Monitor & Improve
Define infrastructure in code	Store in Git repository	Validate, lint & test changes	Provision or update infrastructure	Detect drift, monitor & optimize

Consistency → Repeatability → Scalability → Reliability → Speed

WHY IaC MATTERS

- Eliminate manual errors
- Consistent environments
- Faster provisioning
- Version-controlled changes
- Easy rollbacks
- Audit & compliance
- Scalable operations

1 AWS CLOUDFORMATION

- Native AWS IaC service.
- Uses JSON or YAML templates.
- Manages stacks & resources.
- Handles dependencies automatically.
- Supports drift detection & rollback.

TEMPLATE COMPONENTS

Parameters · Outputs · Resources · Conditions · Mappings · Metadata

BEST PRACTICES

- ☑ Keep templates modular.
- ☑ Use Parameters & Mappings.
- ☑ Enable termination protection.
- ☑ Use Change Sets before update.
- ☑ Tag everything.

2 AWS CDK (CLOUD DEVELOPMENT KIT)

- Define infrastructure using code.
- Supports TypeScript, Python, Java, C#, Go.
- Higher-level abstractions.
- Synthesizes to CloudFormation.
- Great for complex architectures.

CDK FLOW



Write CDK Code → CDK App (Synth) → CloudFormation Template → Deploy (AWS)

BEST PRACTICES

- ☑ Use Constructs for reusability.
- ☑ Use context & configuration.
- ☑ Keep stacks small & focused.
- ☑ Use CDK Pipelines for automation.
- ☑ Apply least privilege IAM.

3 TERRAFORM ON AWS

Terraform

- Vendor-neutral IaC tool.
- Uses HCL (HashiCorp Configuration Language).
- Manages multiple clouds.
- Strong state & plan capabilities.
- Large provider ecosystem.

TERRAFORM WORKFLOW



Write HCL Code → terraform plan → terraform apply

BEST PRACTICES

- ☑ Run terraform fmt & validate.
- ☑ Use modules & variables.
- ☑ Plan before apply.
- ☑ Use remote state.
- ☑ Lock state to prevent conflicts.

4 STATE MANAGEMENT

- Tracks the current state of infrastructure.
- Essential for Terraform.
- Enables collaboration.
- Prevents conflicting changes.
- Stores metadata & dependencies.

BACKENDS (TERRAFORM EXAMPLES)



S3 (State File) · DynamoDB (Locking) · S3 + DDB (Best Practice)

BEST PRACTICES

- ☑ Store state in S3 (encrypted).
- ☑ Use DynamoDB for locking.
- ☑ Enable versioning on S3.
- ☑ Restrict access with IAM.
- ☑ Never commit state to Git.

5 DRIFT DETECTION

- Detects changes made outside of IaC.
- Prevents configuration drift.
- Ensures compliance.
- Available in CloudFormation, AWS Config, and Terraform.

DETECTION METHODS



CloudFormation Drift Detection · AWS Config Rules · Terraform Plan

BEST PRACTICES

- ☑ Schedule regular drift checks.
- ☑ Alert on unauthorized changes.
- ☑ Remediate or update IaC code.
- ☑ Use AWS Config for continuous monitoring.

6 MODULE DESIGN

- Modules = Reusable IaC components.
- Promotes consistency & reduces duplication.
- Encapsulates best practices.
- Easier maintenance & scaling.

GOOD MODULE DESIGN



Single Responsibility · Input Variables · Clear Outputs · Documentation Included · Limited Assumptions

BEST PRACTICES

- ☑ Keep modules small & focused.
- ☑ Use standardized naming.
- ☑ Version your modules.
- ☑ Validate inputs & set defaults.
- ☑ Publish to private registry (if scale).

7 ENTERPRISE AUTOMATION

- Automate the full infrastructure lifecycle.
- Integrate with CI/CD pipelines.
- Enforce policies & compliance.
- Enable self-service for teams.
- Scale across accounts & regions.

AUTOMATION BUILDING BLOCKS



CI/CD Integration · Policy as Code · Multi-Account Strategy · Observability & Alerts · Compliance Checks

BEST PRACTICES

- ☑ Use pipelines for all IaC changes.
- ☑ Enforce guardrails with SCPs & policies.
- ☑ Automate tagging & cost controls.
- ☑ Provide reusable templates & modules.
- ☑ Continuously monitor & improve.

PRO TIP

IaC is not just about writing code— it's about building reliable, scalable, and secure systems automatically.

REMEMBER

Your infrastructure is code. Treat it like application code.

GOLDEN RULE

If it's not in code, it doesn't exist. If it's not versioned, it's risky.

PAGE 13 OF 20

VERIQTA

@veriqta_

AWS SECURITY & COMPLIANCE INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Secure by design. Detect early. Respond fast. Stay compliant. Protect everything.

ENGINEER MINDSET

- ☑ Security is everyone's job
- ☑ Least privilege always
- ☑ Automate security
- ☑ Monitor continuously
- ☑ Prepare for the worst

TOPICS COVERED ON THIS PAGE

1. GuardDuty
2. Security Hub
3. Inspector
4. WAF (Web Application Firewall)
5. Shield (DDoS Protection)
6. AWS Organizations SCPs
7. Compliance Frameworks

AWS SECURITY ARCHITECTURE OVERVIEW



Prevent → Detect → Respond → Recover → Comply → Improve

WHY IT MATTERS

- Prevent breaches
- Detect threats early
- Reduce risk & impact
- Meet compliance requirements
- Protect data & reputation
- Enable secure automation
- Scale securely

1 GUARDDUTY

- Intelligent threat detection service.
- Continuously monitors AWS accounts.
- Uses ML, anomaly detection & threat intelligence.
- Detects malicious behavior.

DETECTS

Malware · Unauthorized Access · Crypto Mining

BEST PRACTICES

- ☑ Enable in all regions.
- ☑ Review findings daily.
- ☑ Integrate with Security Hub.
- ☑ Automate response with EventBridge.

2 SECURITY HUB

- Central security dashboard.
- Aggregates findings from AWS services and partners.
- Prioritizes findings by severity.
- Provides compliance checks.

AGGREGATES FROM

GuardDuty · Inspector · AWS Config · More (Partner Tools)

BEST PRACTICES

- ☑ Enable all relevant standards.
- ☑ Review & prioritize findings.
- ☑ Use insights for remediation.
- ☑ Automate with notifications.

3 INSPECTOR

- Automated security assessment.
- Scans EC2, containers & ECR images.
- Identifies vulnerabilities and misconfigurations.
- Agentless and continuous.

ASSESSMENT TARGETS

EC2 · ECR Images · Lambda · Network

BEST PRACTICES

- ☑ Scan on push for ECR.
- ☑ Schedule regular assessments.
- ☑ Prioritize critical findings.
- ☑ Automate remediation.

4 WAF (WEB APPLICATION FIREWALL)

- Protects web apps from common attacks.
- Filters & monitors HTTP(S) traffic.
- Rule-based & managed rules.
- Custom rules for fine control.

PROTECTS AGAINST

SQL Injection · XSS · Bot Attacks · Bad Bots

BEST PRACTICES

- ☑ Use AWS Managed Rules.
- ☑ Enable logging to CloudWatch.
- ☑ Tune rules to reduce false positives.
- ☑ Regularly update rule sets.

5 SHIELD (DDoS PROTECTION)

- Managed DDoS protection service.
- Shield Standard is always on (free).
- Shield Advanced provides enhancements.
- Detects & mitigates DDoS attacks.

SHIELD OPTIONS

Shield Standard	Shield Advanced
• Always enabled	• Advanced detection
• Automatic mitigation	• Cost protection
• No extra cost	• 24/7 DDoS response team

BEST PRACTICES

- ☑ Enable Shield Advanced for critical apps.
- ☑ Review DDoS metrics & events.
- ☑ Integrate with WAF & CloudFront.
- ☑ Practice with AWS DDoS Response Team.

6 AWS ORGANIZATIONS SCPS

- Service Control Policies set guardrails.
- Apply permissions boundaries to accounts.
- Prevent unwanted or risky actions.
- Help enforce security at scale.

SCP EXAMPLES

Deny Region	Restrict Services	Block Actions
Deny usage of high-risk regions	Allow approved services only	• Deny destructive actions (e.g., Terminate)

HOW SCPS WORK

SCP (Guardrail) → AWS Organizations (Root / OU) → Accounts

BEST PRACTICES

- ☑ Apply SCPs at the OU level.
- ☑ Test SCPs before applying (Deny by default).
- ☑ Use least privilege for allowed actions.
- ☑ Continuously review & update policies.

7 COMPLIANCE FRAMEWORKS

- AWS provides built-in alignment reports.
- Helps meet regulatory requirements.
- Continuously monitored and updated.
- Generate audit-ready reports.

SUPPORTED FRAMEWORKS

- ☑ AWS Foundational Security Best Practices
- ☑ CIS AWS Foundations Benchmark
- ☑ NIST 800-53 Rev. 5
- ☑ PCI DSS
- ☑ ISO 27001
- ☑ SOC 2
- ☑ HIPAA
- ☑ FedRAMP

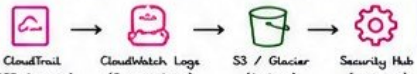
BEST PRACTICES

- ☑ Use AWS Audit Manager for reports.
- ☑ Map controls to your environment.
- ☑ Regularly review compliance status.
- ☑ Automate evidence collection.

SECURITY LAYERS (DEFENSE IN DEPTH)

Identity Layer	→ IAM, Organizations, SCPs
Perimeter Layer	→ WAF, Shield, Security Groups
Workload Layer	→ GuardDuty, Inspector, Antivirus
Data Layer	→ KMS, Encryption, Secrets Manager
Monitoring Layer	→ CloudWatch, CloudTrail, Security Hub

SECURITY LOGGING & AUDITING



BEST PRACTICES

- ☑ Enable CloudTrail in all regions.
- ☑ Centralize logs in a logging account.
- ☑ Protect logs with KMS encryption.
- ☑ Retain logs for compliance needs.

COMMON THREATS

- Credential Compromise
- Privilege Escalation
- Unrestricted Access
- Misconfiguration
- Vulnerabilities
- DDoS Attacks
- Data Exfiltration

INTERVIEW QUICK TIPS

- Explain the service purpose.
- How it detects or prevents.
- Integration with other services.
- How you respond to findings.
- How you ensure compliance.
- Best practices you follow.

PAGE 14 OF 20

VERIQTA

@veriqta_

DISASTER RECOVERY & MULTI-REGION ARCHITECTURE INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Prepare for failure. Design for resilience. Recover with confidence. Stay available.

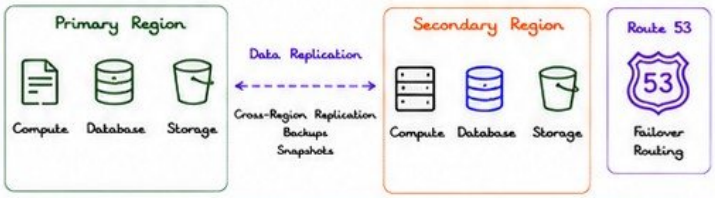
ENGINEER MINDSET

- ☑ Expect failure
- ☑ Design for recovery
- ☑ Test regularly
- ☑ Automate backups
- ☑ Document runbooks
- ☑ Improve continuously

TOPICS COVERED ON THIS PAGE

1. RTO and RPO
2. Backup Strategy
3. Cross-Region Replication
4. Active-Passive Architecture
5. Active-Active Architecture
6. Route 53 Failover
7. Business Continuity

HIGH LEVEL DR ARCHITECTURE OVERVIEW



WHY IT MATTERS

- Minimize downtime
- Protect data
- Meet business SLAs
- Maintain customer trust
- Reduce financial impact
- Meet compliance requirements
- Ensure operational continuity

1 RTO AND RPO

- RTO = Recovery Time Objective
- Maximum acceptable downtime.
- RPO = Recovery Point Objective
- Maximum acceptable data loss.
- Define per application / workload.
- Drive architecture & DR strategy.

EXAMPLE

Application	RTO	RPO
E-Commerce	15 min	5 min
Payment Service	5 min	1 min
Analytics	4 hrs	1 hr

BEST PRACTICES

- ☑ Define RTO/RPO per workload.
- ☑ Align with business impact.
- ☑ Review regularly.
- ☑ Test to validate.

2 BACKUP STRATEGY

- Follow 3-2-1-1-0 backup rule.
- 3 copies of data
- 2 different media
- 1 copy offsite
- 1 copy immutable
- 0 errors (verify backups)
- Use AWS Backup for automation.

BACKUP COMPONENTS



BEST PRACTICES

- ☑ Automate and schedule backups.
- ☑ Encrypt all backups.
- ☑ Test restore regularly.
- ☑ Use lifecycle policies.

3 CROSS-REGION REPLICATION

- Replicate data across regions.
- Amazon S3 Cross-Region Replication
- RDS Read Replicas (Cross-Region)
- Aurora Global Database
- DynamoDB Global Tables
- EBS Snapshots copy
- Use asynchronous replication.

SUPPORTED SERVICES (EXAMPLES)



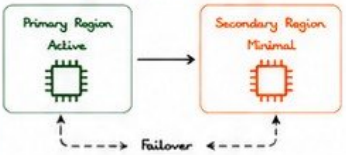
BEST PRACTICES

- ☑ Enable versioning before replication.
- ☑ Monitor replication metrics.
- ☑ Use least privilege IAM roles.
- ☑ Test failover regularly.

4 ACTIVE-PASSIVE ARCHITECTURE

- Primary region handles traffic.
- Secondary region is on standby.
- Lower cost, simpler to manage.
- Failover during disaster.
- Examples: Pilot Light, Warm Standby.

PILOT LIGHT EXAMPLE



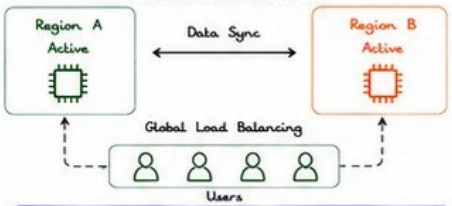
BEST PRACTICES

- ☑ Automate failover runbooks.
- ☑ Keep data in sync.
- ☑ Monitor health of primary region.
- ☑ Test failover regularly.

5 ACTIVE-ACTIVE ARCHITECTURE

- Both regions handle live traffic.
- Higher availability & lower latency.
- More complex & higher cost.
- Requires global data consistency.
- Use when zero downtime is required.

ACTIVE-ACTIVE EXAMPLE



BEST PRACTICES

- ☑ Use Global Database / Global Tables.
- ☑ Use idempotent apps.
- ☑ Handle conflicts gracefully.
- ☑ Monitor performance.

6 ROUTE 53 FAILOVER

- Route traffic based on health checks.
- Failover or failback routing policies.
- Health checks detect failures.
- DNS-based global traffic management.
- Fast, reliable, and highly available.



BEST PRACTICES

- ☑ Use health checks for critical endpoints.
- ☑ Set appropriate TTL values.
- ☑ Test failover scenarios.
- ☑ Document DNS & routing strategy.

7 BUSINESS CONTINUITY

- People, process, technology.
- Document DR plan & runbooks.
- Regular training & testing.
- Communication during incidents.
- Align with business impact analysis.

BUSINESS CONTINUITY PILLARS



CONTINUITY CHECKLIST

- ☑ Business Impact Analysis completed
- ☑ DR plan documented and approved
- ☑ Runbooks tested and updated
- ☑ Stakeholders and contacts defined
- ☑ Regular drills and post-mortems done

PRO TIP

Disaster Recovery is a process, not a destination. Test it or trust it.

REMEMBER

You don't rise to the occasion. You fall to your level of preparation.

GOLDEN RULE

Hope for the best. Prepare for the worst. Design for anything.

PAGE 15 OF 20

VERIQTA

@veriqta_

CLOUD MIGRATION INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Migrate smart. Modernize faster. Reduce risk. Maximize value. Deliver outcomes.

ENGINEER MINDSET

- ☑ Understand before you lift
- ☑ Plan before you migrate
- ☑ Test before you cutover
- ☑ Automate everything
- ☑ Measure results
- ☑ Improve continuously

TOPICS COVERED ON THIS PAGE

1. Migration Strategies
2. AWS Migration Hub
3. Database Migration
4. Application Modernization
5. Hybrid Cloud
6. Cutover Planning
7. Rollback Strategy

CLOUD MIGRATION OVERVIEW

Assess	Plan	Migrate	Optimize	Evolve
Discover, Analyze TCO, Readiness	Strategy, Tools Timeline	Move, Test Validate	Performance Cost, Security	Modernize Innovate

Assess → Plan → Migrate → Optimize → Evolve

WHY IT MATTERS

- Reduce cost & complexity
- Improve performance & scale
- Increase security posture
- Enable innovation & agility
- Support business growth
- Meet compliance requirements
- Maximize ROI

1 MIGRATION STRATEGIES

- Rehost (Lift and Shift)
- Replatform (Lift, Tinker, and Shift)
- Refactor (Re-architect)
- Repurchase (Replace)
- Retain (Do Nothing)
- Retire (Decommission)

6R STRATEGY FRAMEWORK



BEST PRACTICES

- ☑ Choose the right strategy per workload.
- ☑ Start small and iterate.
- ☑ Consider people, process & technology.
- ☑ Align with business goals.

2 AWS MIGRATION HUB

- Central hub for migration tracking.
- Tracks servers, apps, databases.
- Provides tools & best practices.
- Integrates with AWS services.
- Visibility into progress & cost.

MIGRATION HUB WORKFLOW

Discover	Track	Report
AWS Application Discovery Service	Migration Hub Dashboard	Metrics, Cost, Progress

BEST PRACTICES

- ☑ Use MGN for server migrations.
- ☑ Keep inventory data accurate.
- ☑ Update migration plans regularly.
- ☑ Use reports to drive decisions.

3 DATABASE MIGRATION

- Choose the right migration service.
- Minimize downtime & data loss.
- Migrate schema + data + objects.
- Test performance after migration.
- Validate application compatibility.

AWS DATABASE MIGRATION SERVICES

AWS DMS	AWS SCT	AWS DDL
(Online migration)	(Schema conversion)	(Assessment)

BEST PRACTICES

- ☑ Perform trial migrations.
- ☑ Use CDC for low downtime.
- ☑ Optimize post-migration performance.
- ☑ Validate data integrity.

4 APPLICATION MODERNIZATION

- Re-architect for cloud native.
- Use managed services.
- Adopt microservices & containers.
- CI/CD & DevOps transformation.
- Improve observability & security.

MODERNIZATION APPROACHES

Monolith to Microservices	Containers (ECS / EKS)	Serverless (Lambda)

BEST PRACTICES

- ☑ Modernize in small increments.
- ☑ Decouple components.
- ☑ Automate deployments.
- ☑ Measure performance improvements.

5 HYBRID CLOUD

- Extend data center to AWS.
- Use AWS Outposts for local needs.
- Secure connectivity (VPN / DX).
- Consistent identity & access.
- Centralized management & monitoring.

HYBRID CLOUD ARCHITECTURE



BEST PRACTICES

- ☑ Design secure network connectivity.
- ☑ Use IAM Identity Center for access.
- ☑ Monitor across environments.
- ☑ Automate hybrid operations.

6 CUTOVER PLANNING

- Define cutover approach & timeline.
- Data sync & final delta migration.
- Application & system testing.
- Communication & stakeholder updates.
- Go/No-Go decision criteria.

CUTOVER TIMELINE EXAMPLE

Prep (Weeks)	Pre-Cutover (Days)	Cutover (Window)	Post-Cutover (Validation)

BEST PRACTICES

- ☑ Schedule during low business impact.
- ☑ Have clear roles & responsibilities.
- ☑ Automate steps where possible.
- ☑ Validate every component.

7 ROLLBACK STRATEGY

- Always have a rollback plan.
- Maintain system snapshots.
- Retain source environment until stable.
- Document rollback procedures.
- Test rollback before cutover.

ROLLBACK STRATEGY FLOW

Issue Detected (Post-Cutover)	Trigger Rollback	Return to Previous State	Verify & Communicate

BEST PRACTICES

- ☑ Keep rollback simple & tested.
- ☑ Automate backups & snapshots.
- ☑ Define RTO & RPO for rollback.
- ☑ Communicate clearly with all teams.

MIGRATION PRINCIPLE

You don't migrate to the cloud.
You migrate to better outcomes.
Business first. Cloud second.

REMEMBER

A successful migration is not lifting workloads.
It's delivering business value.

GOLDEN RULE

Test. Validate. Communicate.
Plan for failure.
Design for recovery.

PAGE 16 OF 20

VERIQTA

@veriqta_

ENTERPRISE AWS SYSTEM DESIGN INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Design for scale. Build for security. Operate with excellence. Govern everywhere. Automate everything.

ENGINEER MINDSET

- ☑ Think in systems
- ☑ Plan for growth
- ☑ Design for failure
- ☑ Automate guardrails
- ☑ Measure everything
- ☑ Improve continuously

TOPICS COVERED ON THIS PAGE

1. Multi-account Architecture
2. Landing Zone
3. Shared Services
4. Network Hub
5. Logging Account
6. Security Account
7. Enterprise Governance

ENTERPRISE AWS REFERENCE ARCHITECTURE



Landing Zone	Security Account	Network Hub	Shared Services	Workload Accounts
Accounts	GuardDuty	Transit Gateway	AD / Identity	Dev / Test / Prod
Network	Security Hub	DNS (Route 53)	CI/CD Tools	Applications
Guardrails	IAM Identity Center	VPN / DX	Artifacts	Data

Centralized Logging & Monitoring

CloudTrail • CloudWatch • Config • VPC Flow Logs • S3 • Athena • QuickSight

WHY IT MATTERS

- Enable scale & agility
- Improve security posture
- Reduce operational overhead
- Lower cost with control
- Ensure compliance
- Standardize & automate
- Support business growth

1 MULTI-ACCOUNT ARCHITECTURE

- Use AWS Organizations for management.
- Separate workloads by environment.
- Isolate by business unit or application.
- Apply guardrails with SCPs.
- Centralize billing and governance.

EXAMPLE STRUCTURE



BEST PRACTICES

- ☑ Use dedicated accounts for isolation.
- ☑ Least privilege access across accounts.
- ☑ Automate account provisioning.
- ☑ Tag resources consistently.

2 LANDING ZONE

- Pre-configured, secure AWS environment.
- Baseline accounts, networking, and guardrails.
- Automated using AWS Control Tower.
- Provides a consistent starting point.
- Enforces best practices by default.

LANDING ZONE COMPONENTS



BEST PRACTICES

- ☑ Use Control Tower or custom LZ.
- ☑ Enable MFA & least privilege.
- ☑ Apply mandatory tags.
- ☑ Continuously validate guardrails.

3 SHARED SERVICES

- Centralize common services.
- Reduce duplication and cost.
- Provide reusable capabilities.
- Improve consistency and control.
- Examples: AD, CI/CD, DNS, IAM.

COMMON SHARED SERVICES



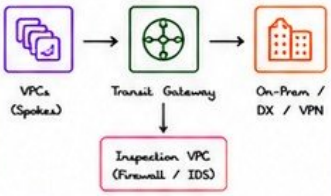
BEST PRACTICES

- ☑ Offer as a service (self-service).
- ☑ Define SLAs for shared services.
- ☑ Secure and monitor shared services.
- ☑ Chargeback or showback usage.

4 NETWORK HUB

- Centralize connectivity.
- Use AWS Transit Gateway.
- Connect VPCs, on-prem, and regions.
- Inspect traffic with firewall.
- Simplify routing and security.

NETWORK HUB DESIGN



BEST PRACTICES

- ☑ Use TGW for scalable routing.
- ☑ Centralize egress & inspection.
- ☑ Segment with route tables.
- ☑ Enable DNS resolution strategy.

5 LOGGING ACCOUNT

- Centralize logs from all accounts.
- Store logs securely and immutably.
- Use S3, CloudTrail, CloudWatch Logs.
- Enable analysis and auditing.
- Support compliance and forensics.

LOGGING ARCHITECTURE



BEST PRACTICES

- ☑ Enable CloudTrail org-wide.
- ☑ Use S3 with S3 Object Lock.
- ☑ Partition logs by account/region/date.
- ☑ Monitor log delivery & integrity.

6 SECURITY ACCOUNT

- Central place for security services.
- Aggregate findings and alerts.
- Enforce policies and guardrails.
- Protect workloads across accounts.
- Drive security operations.

KEY SECURITY SERVICES



BEST PRACTICES

- ☑ Aggregate all security findings.
- ☑ Automate responses with EventBridge.
- ☑ Enable org-wide Config rules.
- ☑ Use least privilege roles.

7 ENTERPRISE GOVERNANCE

- Define policies, standards, and processes.
- Use Organizations, SCPs, and Tag Policies.
- Enforce cost, security, and compliance.
- Automate governance with IaC.
- Review and improve continuously.

GOVERNANCE PILLARS



BEST PRACTICES

- ☑ Define clear naming & tagging standards.
- ☑ Automate compliance checks.
- ☑ Regular access reviews.
- ☑ Measure and report governance KPIs.

ENTERPRISE AWS PRINCIPLES

- ☆ Secure by design.
- ☆ Operate with automation.
- ☆ Design for failure.
- ☆ Use data to drive decisions.
- ☆ Continuously evolve.

KEY AWS SERVICES USED



INTERVIEW QUICK TIPS

- Always start with requirements.
- Explain trade-offs clearly.
- Consider security & cost.
- Design for scale and failure.
- Communicate assumptions.

PAGE 17 OF 20

VERIQTA

@veriqta_

PRODUCTION FAILURE SCENARIOS

INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Expect failure. Design for resilience. Detect early. Respond fast. Recover smarter.

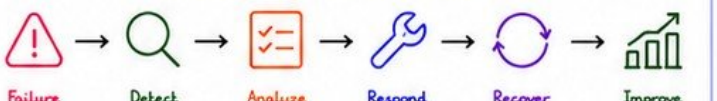
ENGINEER MINDSET

- ☑ Systems will fail.
- ☑ Design for failure.
- ☑ Automate detection.
- ☑ Practice recovery.
- ☑ Learn from every incident.
- ☑ Improve continuously.

TOPICS COVERED ON THIS PAGE

1. Region Outage
2. AZ Failure
3. EC2 Failures
4. S3 Availability Issues
5. RDS Failover
6. Network Failures
7. Incident Response Workflow

PRODUCTION RESILIENCE OVERVIEW



Resilient Systems = Fast Detection + Quick Response + Full Recovery + Continuous Improvement

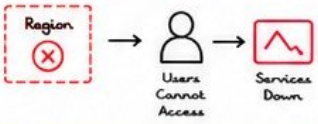
WHY IT MATTERS

- Minimize downtime & impact
- Protect customer trust
- Meet SLA & compliance
- Reduce financial loss
- Ensure data integrity
- Strengthen system resilience
- Improve incident readiness

1 REGION OUTAGE

- Complete region becomes unavailable.
- Causes: Power loss, network, disasters.
- All AZs in the region are affected.
- Impact: Full service downtime.

EXAMPLE IMPACT



BEST PRACTICES

- ☑ Use Multi-Region architecture.
- ☑ Use Route 53 health checks.
- ☑ Replicate data across regions.
- ☑ Test DR regularly.

2 AZ FAILURE

- Single Availability Zone is down.
- Causes: Hardware, power, network.
- Resources in the AZ are affected.
- Impact: Partial service degradation.

EXAMPLE IMPACT



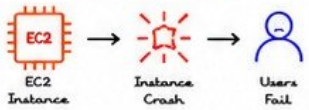
BEST PRACTICES

- ☑ Deploy across multiple AZs.
- ☑ Use Auto Scaling Groups.
- ☑ Use ELB / ALB across AZs.
- ☑ Avoid single-AZ dependencies.

3 EC2 FAILURES

- Instance crashes or becomes unhealthy.
- Causes: Hardware, kernel panic, OOM, user error.
- Impact: App errors, downtime.

EXAMPLE IMPACT



BEST PRACTICES

- ☑ Use Auto Scaling & health checks.
- ☑ Use status checks (2/2).
- ☑ Store data outside EC2.
- ☑ Implement self-healing.

4 S3 AVAILABILITY ISSUES

- S3 slow, errors, or eventual consistency delays.
- Causes: Region issue, throttling, network.
- Impact: Failed uploads / downloads.

EXAMPLE IMPACT



BEST PRACTICES

- ☑ Use transfer acceleration if needed.
- ☑ Implement retries & backoff.
- ☑ Enable versioning & lifecycle.
- ☑ Monitor with CloudWatch.

5 RDS FAILOVER

- Primary DB becomes unavailable.
- Triggers automatic or manual failover.
- Causes: Hardware, AZ issue, engine crash.
- Impact: Connection failures, latency.

FAILOVER FLOW



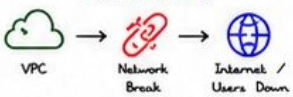
BEST PRACTICES

- ☑ Use Multi-AZ or Multi-Region.
- ☑ Test failover regularly.
- ☑ Monitor replication lag.
- ☑ Use read replicas correctly.

6 NETWORK FAILURES

- VPC, Transit Gateway, or Internet connectivity issues.
- Causes: Route errors, BGP, DX failure, NACL/SG misconfig.
- Impact: App unreachable, timeouts.

EXAMPLE IMPACT



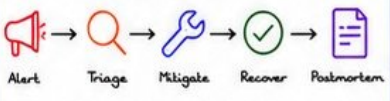
BEST PRACTICES

- ☑ Design redundant network paths.
- ☑ Use Route 53 failover / health checks.
- ☑ Monitor network metrics.
- ☑ Document network topology.

7 INCIDENT RESPONSE WORKFLOW

- Structured process to handle incidents.
- Follow playbooks and runbooks.
- Communicate clearly.
- Focus on users, data, and recovery.

RESPONSE PHASES



BEST PRACTICES

- ☑ Use runbooks & automation.
- ☑ Define roles & responsibilities.
- ☑ Keep stakeholders informed.
- ☑ Review & improve after every incident.

INCIDENT IMPACT LEVELS

SEV 1	Critical	Complete outage, major impact
SEV 2	High	Partial outage, major feature down
SEV 3	Medium	Degraded performance, minor impact
SEV 4	Low	Minimal impact, advisory

COMMUNICATION CHANNELS

- Slack / Teams
- Email / SMS
- Status Page
- Conference Bridge

KEY AWS SERVICES FOR RESILIENCE

- Route 53 (Failover)
- CloudWatch (Monitoring)
- Auto Scaling (Self-Healing)
- S3 CRR (Cross-Region)
- RDS Multi-AZ (High Availability)
- AWS Backup (Backups)
- Lambda (Event Response)

MONITORING METRICS TO WATCH

- CPU Utilization
- Status Check Failed
- 5xx Error Rate
- Latency (p95)
- Replication Lag
- Network In / Out
- Disk Queue Depth
- S3 4xx/5xx Errors

PRO TIP

You can't prevent all failures, but you can prepare, detect, and recover like a pro.

REMEMBER

The goal is not zero incidents. The goal is zero unhandled incidents.

GOLDEN RULE

Automate the boring. Monitor everything. Improve every day.

PAGE 18 OF 20

VERIQTA

AWS TROUBLESHOOTING MASTERCLASS

INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Investigate like a detective. Think like an engineer. Solve like a senior. Recover like a pro.

@veriqta_

ENGINEER MINDSET

- ☑ Don't guess. Verify.
- ☑ Check one layer at a time.
- ☑ Measure before you act.
- ☑ Correlate logs & metrics.
- ☑ Find the root cause.
- ☑ Fix permanently.

TOPICS COVERED ON THIS PAGE

1. End-to-end Investigation
2. AWS CLI Debugging
3. CloudWatch Analysis
4. Log Correlation
5. Network Diagnostics
6. Root Cause Analysis
7. Production Recovery

END-TO-END TROUBLESHOOTING FRAMEWORK

1. Identify	2. Collect	3. Analyze	4. Isolate	5. Resolve	6. Validate	7. Improve
Understand the problem	Gather logs, metrics, data	Find patterns & clues	Narrow down the cause	Fix the issue or mitigate	Confirm the issue is fixed	Document & prevent repeat

Observe → Collect → Correlate → Hypothesize → Test → Resolve → Verify → Improve

WHY IT MATTERS

- Reduce downtime
- Improve mean time to resolution
- Prevent recurring issues
- Protect customer experience
- Lower cost and risk
- Build reliability & trust
- Make data-driven decisions

1 END-TO-END INVESTIGATION

Follow a structured approach.

- Define the problem clearly.
- Identify the scope and impact.
- Walk the request flow.
- Check all layers and dependencies.
- Validate assumptions.

INVESTIGATION LAYERS

User / Client
↓
DNS / Route 53
↓
Network / VPC
↓
Load Balancer / CDN
↓
Application / Compute
↓
Database / Storage
↓
Monitoring / Logs

BEST PRACTICES

- ☑ Start broad, then narrow.
- ☑ Change one variable at a time.
- ☑ Document every finding.
- ☑ Communicate with stakeholders.
- ☑ Verify the fix with proof.

2 AWS CLI DEBUGGING

Use CLI to verify, inspect & debug.

ESSENTIAL CLI COMMANDS

EC2
```
aws ec2 describe-instances
aws ec2 describe-status-checks
aws ec2 describe-snapshots
```

S3
```
aws s3 ls s3://bucket-name
aws s3api head-object
aws s3api list-object-versions
```

RDS
```
aws rds describe-db-instances
aws rds describe-events
aws rds describe-db-clusters
```

CloudWatch
```
aws cloudwatch get-metric-statistics
aws logs filter-log-events
aws cloudwatch describe-alarms
```

General
```
aws sts get-caller-identity
aws configure list
aws service-quotas list-service-quotas
```

BEST PRACTICES

- ☑ Use --region explicitly.
- ☑ Use --utput table for readability.
- ☑ Pipe output with jq for filtering.
- ☑ Script repeatable checks.
- ☑ Use profiles for multi-account.

3 CLOUDWATCH ANALYSIS

Metrics, alarms & dashboards.

KEY COMPONENTS

Metrics	CPU, Memory, Errors, Latency
Alarms	Thresholds trigger actions
Dashboards	Visualize what matters
Events	System & service events

USEFUL CLI COMMANDS

```
aws cloudwatch list-metrics
aws cloudwatch get-metric-statistics
aws cloudwatch describe-alarms
aws cloudwatch get-dashboard
```

4 LOG CORRELATION

Correlate logs across services.

LOG SOURCES

- CloudWatch Logs — Application & system logs
- VPC Flow Logs — Network traffic logs
- ELB / ALB Logs — Request & response logs
- S3 Server Access Logs — Access & error logs
- AWS X-Ray — Distributed tracing

BEST PRACTICES

- ☑ Use consistent timestamps (UTC).
- ☑ Add correlation IDs in apps.
- ☑ Centralize logs to one account.
- ☑ Use Log Insights queries.
- ☑ Create saved queries & alerts.

5 NETWORK DIAGNOSTICS

Validate connectivity & networking.

DIAGNOSTIC TOOLS

VPC Reachability Analyzer · Ping / Curl / Dig DNS (from EC2) · VPC Flow Logs · Route Tables & NACLs

COMMON NETWORK ISSUES

- No route to destination
- NACL blocking traffic
- Security Group misconfiguration
- DNS resolution failure
- VPC Endpoint / NAT Gateway issues

USEFUL CLI COMMANDS

```
aws ec2 describe-route-tables
aws ec2 describe-security-groups
aws ec2 describe-network-acls
aws ec2 describe-vpc-endpoints
aws ec2 describe-flow-logs
```

BEST PRACTICES

- ☑ Draw the network diagram.
- ☑ Test from inside the VPC.
- ☑ Check both ingress & egress.
- ☑ Use Reachability Analyzer.
- ☑ Enable Flow Logs in all VPCs.

6 ROOT CAUSE ANALYSIS

Go beyond symptoms.

RCA TECHNIQUES

- 5 Whys — Dig until the real cause.
- Fishbone Diagram — Map categories of causes
- Fault Tree Analysis — Break down failure paths
- Timeline Analysis — Reconstruct the sequence
- Data Correlation — Logs + Metrics + Traces

RCA CHECKLIST

- ☑ What happened?
- ☑ When did it start?
- ☑ What changed?
- ☑ Why did it fail?
- ☑ What is the real root cause?
- ☑ How do we prevent it?

BEST PRACTICES

- ☑ Avoid blaming individuals.
- ☑ Focus on system & process.
- ☑ Document evidence.
- ☑ Share learnings.
- ☑ Share learning ideas.
- ☑ Implement preventive actions.

7 PRODUCTION RECOVERY

Recover fast. Recover safe.

RECOVERY STRATEGIES

Mitigate	Workaround	Rollback	Failover	Restore
Reduce impact quickly	Temporary solution	Revert to last good state	Switch to healthy region/ AZ	From backup / snapshot

RECOVERY CHECKLIST

- ☑ Confirm the issue and impact.
- ☑ Communicate to stakeholders.
- ☑ Apply mitigation or workaround.
- ☑ Validate service recovery.
- ☑ Monitor closely after recovery.
- ☑ Document the incident.

BEST PRACTICES

- ☑ Have runbooks ready.
- ☑ Practice game days.
- ☑ Automate recovery steps.
- ☑ Keep backups tested.
- ☑ Learn and improve continuously.

TROUBLESHOOTING COMMAND CHEAT SHEET

Purpose	Command (Example)
Check Instance Status	aws ec2 describe-instance-status
Check RDS Events	aws rds describe-events
Check ALB Target Health	aws elbv2 describe-target-health
Check S3 Bucket	aws s3api head-bucket
Check IAM Identity	aws sts get-caller-identity
Check CloudWatch Alarms	aws cloudwatch describe-alarms
View Recent Logs	aws logs filter-log-events

TROUBLESHOOTING MANTRAS

- ★ Verify before you modify.
- ★ Narrow the scope.
- ★ Trust the data.
- ★ Correlate, don't guess.
- ★ Fix the cause, not the symptom.

TRUST-THE-DATA

COMMON TOOLS STACK

CloudWatch (Metrics & Logs) · X-Ray (Tracing) · Systems Manager (Ops) · GuardDuty (Security) · Config (Compliance) · Trusted Advisor (Best Practices)

INTERVIEW QUICK TIPS

- Explain your process clearly.
- Talk about trade-offs.
- Show CLI & logging knowledge.
- Mention automation.
- Share real incident examples.

PAGE 19 OF 20

VERIQTA

AWS ENTERPRISE CHEAT SHEET

INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

One page. Maximum value. Command ready. Architect ready. Production ready. You ready.

@veriqta_

ENGINEER MINDSET

- [x] Automate everything
- [x] Document everything
- [x] Secure by design
- [x] Monitor everything
- [x] Analyze continuously
- [x] Improve continuously

AWS CLI ESSENTIAL COMMANDS

```
aws configure                          # Configure credentials
aws sts get-caller-identity            # Who am I?
aws --version                          # Check CLI version
aws help                               # All CLI commands
aws <service> help                     # Service help
aws <service> list-*                   # List resources
aws <service> describe-*               # Detailed info
aws <service> create-*                 # Create resource
aws <service> delete-*                 # Delete resource
aws <service> update-*                 # Update resource
aws ec2 describe-instances --query
'Reservations[*].Instances[*].InstanceId' --output table
                                       # Output control
aws configure list                     # View current config
aws configure list-profiles            # List profiles
```

PRO TIP: Use --region, --profile, --output json|table|text
Example: aws ec2 describe-instances --region us-east-1 --output table

EC2 CLI COMMANDS

```
aws ec2 describe-instances             # List instances
aws ec2 start-instances --instance-ids <id>   # Start instance
aws ec2 stop-instances --instance-ids <id>    # Stop instance
aws ec2 reboot-instances --instance-ids <id>  # Reboot instance
aws ec2 terminate-instances --instance-ids <id>  # Terminate
aws ec2 describe-instances --filters   # Filter instances
aws ec2 describe-instance-status       # Status checks
aws ec2 describe-security-groups       # List SGs
aws ec2 describe-key-pairs             # List key pairs
aws ec2 describe-volumes               # EBS volumes
aws ec2 create-snapshot --volume-id <vol-id>  # Create snapshot
aws ec2 describe-images --owners self  # AMIs
aws ec2 describe-availability-zones    # AZs
aws ec2 describe-vpcs                  # List VPCs
```

S3 CLI COMMANDS

```
aws s3 ls                              # List buckets
aws s3 ls s3://bucket-name             # List objects
aws s3 cp file.txt s3://bucket/        # Upload file
aws s3 cp s3://bucket/file.txt .       # Download file
aws s3 sync ./folder s3://bucket/      # Sync folder
aws s3 rm s3://bucket/file.txt         # Delete object
aws s3 rm s3://bucket/ --recursive     # Delete all
aws s3 mb s3://new-bucket              # Make bucket
aws s3 rb s3://bucket-name             # Remove bucket
aws s3api head-bucket --bucket bucket
                                       # Bucket exists?
aws s3api put-bucket-versioning \
  --bucket bucket --versioning-configuration
Status=Enabled                         # Enable versioning
aws s3api put-bucket-encryption --bucket bucket
  --server-side-encryption-configuration
                                       # Enable encryption
```

IAM CLI COMMANDS

```
aws iam list-users                     # List users
aws iam get-user --user-name <user>    # Get user
aws iam create-user --user-name <user> # Create user
aws iam delete-user --user-name <user> # Delete user
aws iam list-groups                    # List groups
aws iam add-user-to-group \
  --user-name <user> --group-name <group>  # Add user to group
aws iam attach-user-policy \
  --user-name <user> --policy-arn <arn>    # Attach policy
aws iam list-attached-user-policies \
  --user-name <user>                   # List policies
aws iam list-roles                     # List roles
aws iam get-role --role-name <role>    # Get role
aws iam list-policies                  # List policies
aws iam simulate-principal-policy \
  --policy-source-arn <arn> --action-names
ec2:DescribeInstances                  # Simulate
```

VPC CLI COMMANDS

```
aws ec2 describe-vpcs                  # List VPCs
aws ec2 create-vpc --cidr-block 10.0.0.0/16   # Create VPC
aws ec2 describe-subnets               # List subnets
aws ec2 create-subnet --vpc-id <vpc-id> --cidr-block
    10.0.1.0/24 --availability-zone us-east-1a  # Create subnet
aws ec2 describe-route-tables          # List route tables
aws ec2 create-route-table --vpc-id <vpc-id>  # Create route table
aws ec2 create-route --route-table-id <rtb-id> \
  --destination-cidr-block 0.0.0.0/0 --gateway-id <igw-id>
                                       # Add route
aws ec2 associate-route-table --subnet-id <subnet-id>
  --route-table-id <rtb-id>            # Associate
aws ec2 describe-internet-gateways     # List IGWs
aws ec2 create-internet-gateway        # Create IGW
aws ec2 attach-internet-gateway
  --internet-gateway-id <igw-id> --vpc-id <vpc-id>   # Attach IGW
aws ec2 describe-nat-gateways          # List NAT GWs
aws ec2 create-nat-gateway --subnet-id <subnet-id> \
  --allocation-id <eip-alloc-id>
                                       # Create NAT GW
```

6 TROUBLESHOOTING WORKFLOW

1. **IDENTIFY** — What is broken? Who is affected? When did it start?
2. **COLLECT** — Collect logs, metrics, events, and data. Use CloudWatch, CLI, Logs.
3. **ANALYZE** — Find patterns. Compare with baseline. Form a hypothesis.
4. **ISOLATE** — Narrow down the blast radius. Eliminate unrelated factors. Find the cause.
5. **RESOLVE** — Apply fix or mitigation. Validate the solution. Restore service.
6. **VERIFY** — Confirm full recovery. Monitor closely. Ensure no recurrence.
7. **DOCUMENT** — Write postmortem. Lessons learned. Improve process.

ARCHITECTURE CHECKLIST

RELIABILITY
- [x] Multi-AZ / Multi-Region
- [x] Auto Scaling configured
- [x] Health checks enabled
- [x] Backups and snapshots
- [x] Failover strategy defined

PERFORMANCE
- [x] Right instance types
- [x] Load balancing
- [x] Caching strategy
- [x] Database optimized
- [x] CDN for global delivery

SECURITY
- [x] Least privilage IAM
- [x] Encryption in transit & at rest
- [x] Security groups / NACLs
- [x] WAF / Shield protection
- [x] Secrets in AWS Secrets Manager

OPERATIONS
- [x] CloudWatch monitoring
- [x] Alarms & notifications
- [x] Centralized logging
- [x] Automation (IaC)
- [x] Runbooks & playbooks

COST OPTIMIZATION
- [x] Right sizing
- [x] Reserved / Savings Plans
- [x] S3 lifecycle policies
- [x] Unused resources removed
- [x] Cost monitoring & budgets

COMPLIANCE
- [x] Logging & auditing
- [x] Compliance frameworks
- [x] Config rules enabled
- [x] Tagging strategy
- [x] Data residency

COMMON AWS CLI OUTPUT OPTIONS

--output json	Structured output
--output table	Readable table
--output text	Plain text
--output yaml	YAML format
--query <JMESPath>	Filter specific data

EXAMPLE:

```
aws ec2 describe-instances \
  --query 'Reservations[*].Instances[*].[ID:
        InstanceId,State:State.Name]' \
--output table
```

GLOBAL OPTIONS (USE EVERYWHERE)

--region <region>	Specify region
--profile <name>	Use specific profile
--output <type>	Output format
--query <JMESPath>	Filter output
--no-paginate	Disable pagination
--debug	Detailed debug logs
--cli-read-timeout <sec>	Read timeout
--cli-connect-timeout <sec>	Connect timeout
--endpoint-url <url>	Custom endpoint

USEFUL AWS SERVICES (QUICK REFERENCE)

- EC2 — Compute
- S3 — Storage
- RDS — Database
- VPC — Networking
- IAM — Identity
- CloudWatch — Monitoring
- CloudTrail — Audit & Logs
- Route 53 — DNS
- Lambda — Serverless
- DynamoDB — NoSQL DB

QUICK TIPS

- ☆ Use CLI for automation.
- ☆ Use IAM roles, not users.
- ☆ Tag everything.
- ☆ Monitor everything.
- ☆ Backup everything.
- ☆ Test your DR plan.
- ☆ Document your architecture.
- ☆ Keep learning every day.

Stay consistent.
Stay curious.
Stay senior.

REMEMBER: The best engineers don't memorize commands. They understand systems.

Think. Automate. Secure. Scale.
That's how we build in the cloud.

VERIQTA

@veriqta_

ENTERPRISE CLOUD ENGINEER FINAL ASSESSMENT

INTERVIEW QUESTIONS

SENIOR INTERVIEW TOPICS

Architect for scale. Secure by design. Automate everything. Operate with excellence. Deliver value.

ENGINEER MINDSET

- ☑ Think like an architect
- ☑ Solve real problems
- ☑ Automate everything
- ☑ Secure by design
- ☑ Measure & improve
- ☑ Own the outcome

1 AWS READINESS

- ☑ Account & billing
- ☑ IAM & access
- ☑ AWS Organizations
- ☑ Guardrails in place
- ☑ Tagging strategy
- ☑ Cost management
- ☑ Monitoring enabled
- ☑ Backup strategy

Score /10

2 CLOUD ARCHITECTURE

- ☑ Well-Architected
- ☑ Scalable design
- ☑ Resilient by design
- ☑ High availability
- ☑ Decoupled services
- ☑ Performance optimized
- ☑ Secure architecture
- ☑ Operational excellence

Score /10

3 INFRASTRUCTURE DESIGN

- ☑ VPC & networking
- ☑ Compute design
- ☑ Storage strategy
- ☑ Database design
- ☑ Load balancing
- ☑ Auto scaling design
- ☑ IaC implementation
- ☑ Environment strategy

Score /10

4 SECURITY READINESS

- ☑ Identity & access
- ☑ Network security
- ☑ Data protection
- ☑ Encryption strategy
- ☑ Logging & audit
- ☑ Threat detection
- ☑ Compliance readiness
- ☑ Incident response plan

Score /10

5 TROUBLESHOOTING ASSESSMENT

- ☑ Issue identification
- ☑ Logs & metrics use
- ☑ Root cause analysis
- ☑ Service knowledge
- ☑ AWS CLI skills
- ☑ Problem resolution
- ☑ Communication
- ☑ Post-incident review

Score /10

6 SYSTEM DESIGN EVALUATION

- ☑ Requirements clarity
- ☑ Architecture diagram
- ☑ Scaling strategy
- ☑ High availability
- ☑ Data flow design
- ☑ Failure handling
- ☑ Cost optimization
- ☑ Trade-off analysis

Score /10

7 PRODUCTION ENGINEER SCORECARD

- ☑ Ownership mindset
- ☑ Automation focus
- ☑ Reliability driven
- ☑ Observability first
- ☑ Customer impact
- ☑ Continuous learning
- ☑ Documentation
- ☑ Collaboration

Score /10

ASSESSMENT SUMMARY

	CATEGORY	SCORE	MAX	%
1	AWS Readiness		/10	
2	Cloud Architecture		/10	
3	Infrastructure Design		/10	
4	Security Readiness		/10	
5	Troubleshooting		/10	
6	System Design		/10	
7	Production Engineer		/10	
	TOTAL		/70	%

OVERALL READINESS LEVEL

0 – 34	35 – 49	50 – 59	60 – 70
Needs Improvement	Developing Engineer	Strong Engineer	Production Ready

GOAL:
Score 60/70 or higher to be production ready.

FINAL VERDICT

- ☐ Production Ready
- ☐ Strong Engineer
- ☐ Developing Engineer
- ☐ Needs Improvement

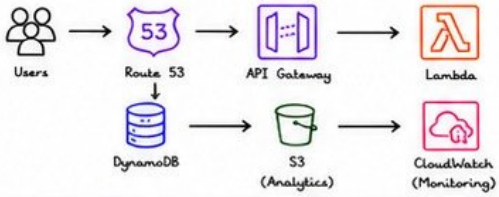
COMMENTS

SCENARIO BASED QUESTIONS

- Design a multi-region, highly available application for millions of users.
- How would you secure an AWS environment for a regulated industry?
- Your AWS bill increased 300% suddenly. How do you investigate?
- RDS database is down in production. What do you do?
- A critical application is running slow. How do you troubleshoot?

SYSTEM DESIGN CHALLENGE (EXAMPLE)

Design a serverless URL shortener that handles 100M requests per day.



What to evaluate:

- ☑ Scalability
- ☑ High Availability
- ☑ Security
- ☑ Cost Optimization
- ☑ Operational Excellence

PRODUCTION READINESS CHECKLIST

- ☑ Architecture reviewed
- ☑ Security controls enabled
- ☑ Monitoring & alerting in place
- ☑ Backups tested
- ☑ Disaster recovery validated
- ☑ Performance & load tested
- ☑ Cost optimized
- ☑ IAM least privilege
- ☑ Logging centralized
- ☑ Runbooks documented
- ☑ Automation implemented
- ☑ Incident response tested
- ☑ Compliance validated
- ☑ Stakeholders aligned

FINAL TIPS

- Understand the WHY behind every service.
- Think in systems, not services.
- Communicate clearly and confidently.
- Own problems and drive solutions.
- Keep learning. Stay curious. Build. Break. Learn. Repeat.

ENGINEER MANTRAS

- ★ There is no magic, only understanding.
- ★ Automation is a force multiplier.
- ★ Good engineers fix problems.
- ★ Great engineers prevent them.
- ★ Build for failure. Design for scale.

TRUST – THE – PROCESS

REMEMBER

- ★ You don't need to know everything.
- ★ You need the right mindset.
- ★ You need the right process.
- ★ You need the right attitude.

Be reliable. Be valuable. Be irreplaceable.

BUILD • OPERATE • OPTIMIZE • REPEAT